
Minimizing Mempool Dependency in PoW Mining on Blockchain: A Paradigm Shift with Compressed Block Representation for Enhanced Scalability, Decentralization and Security.

Gyu Chol Kim

Faculty of Information Science and Technology, Kim Chaek University of Technology, Democratic People's Republic of Korea

Corresponding author: kcu201104@163.com , kgc841110@star-co.net.kp

ABSTRACT While existing Proof-of-Work (PoW) based blockchain protocols have demonstrated innovative potential, they face inherent limitations regarding scalability, efficiency, and decentralization. The compact block propagation method, though effective in reducing network bandwidth and propagation delay in ideal environments, suffers from performance degradation due to mempool inconsistencies among nodes. This paper proposes a novel block propagation and consensus protocol that mitigates the blockchain's dependency on mempool synchronization. The proposed approach redefines the PoW process to shorten the time to consensus despite increased block sizes. Specifically, it includes a compressed transaction input ID list within the compact block to induce nodes to immediately begin mining without full verification. The full verification of transactions adopts a 'delayed verification' method, performed in parallel with the mining operation. This study enables the processing of more transactions quickly while maintaining the decentralization and security of Bitcoin (e.g., achieving approximately 66.7 TPS with 10MB blocks).

Keyword: Blockchain, Compact Block, Decentralization, Delayed Verification, Proof-of-Work (PoW)

1. INTRODUCTION

The scalability of blockchain technology has been a primary focal point of global research in distributed ledger technology for over a decade [1-7]. While Bitcoin, proposed by Satoshi Nakamoto [8], established a foundation of decentralized trust through Proof-of-Work (PoW) and the Longest Chain Rule [9], it left behind the challenge of physical limits on transaction throughput (TPS). To address this, previous researchers have introduced innovative solutions from various perspectives. We build upon these contributions while proposing a new paradigm to overcome their inherent limitations.

The scalability of PoW blockchains has been a central research theme, with numerous scholars proposing diverse solutions over the past decade [1-3, 10, 11].

First, extensive research into Layer 2 and off-chain solutions has been conducted to increase throughput without directly modifying the base layer (Layer 1) [12]. State channel methods, such as the Lightning Network [13], decentralized traffic by processing microtransactions off-chain, while Rollup technologies sought to ensure data availability by compressing and recording transaction data on the main chain. These studies achieved the monumental feat of reducing the burden on the main chain while maintaining its security. However, they introduce secondary risks, including increased complexity in user experience and the intervention of centralized sequencers.

Layer 2 solutions like the Lightning Network and Rollups have significantly offloaded transaction pressure but introduce UX complexity and potential centralization risks [12]. Concurrently, research into data compression and network optimization (Light-net/Propagation) has followed, aiming to improve throughput by enhancing the propagation efficiency of the main chain. BIP 152 (Compact Blocks) [14] drastically reduced block propagation size by recycling transactions within the mempool, and research such as Graphene [15] achieved propagation efficiency near theoretical limits by introducing Invertible Bloom Lookup Tables (IBLT).

Furthermore, physical layer optimizations, such as the FIBRE block relay network [16], have significantly contributed to reducing propagation latency. These pioneering works have been instrumental in lowering the physical barriers of block propagation.

Innovative propagation technologies such as Compact Blocks and Graphene have minimized bandwidth usage but remain tethered to the serial verification model. Despite these remarkable advancements, a fundamental bottleneck remains: the "Serial Verification" structure. Previous compression technologies focused primarily on reducing block "size" but failed to resolve the "temporal dependency", which is the requirement that mining for block $n + 1$ must halt until the propagation and verification of block n are complete. In an era of large-scale blocks, this serial structure forces honest miners into "idle time," which inevitably increases the orphan block rate and erodes the network's security budget. Despite advancements in compression, the inherent serial bottleneck where mining $n + 1$ must wait for block n continues to restrict throughput and increase orphan rates. Building upon the technological foundations laid by previous researchers, this paper presents a more fundamental solution: the "asynchronization of the consensus structure," moving beyond mere propagation efficiency. The proposed scheme completely hides propagation latency within the PoW computation time (Latency Hiding) through a post-confirmation mechanism where block $n + k$ validates block n . Furthermore, by implementing a UTXO Freezing (Mempool Freezing) technique we have achieved a "logical isolation" that prevents the invalidation of a specific block from leading to the collapse of the entire chain.

Our research proposes a paradigm shift towards asynchronous verification and logical isolation, hiding latency within the PoW process to break the serial bottleneck once and for all. Respecting the dedicated research directions of our predecessors, this study proposes a simple yet definitive asynchronous protocol to achieve the goal of a "high-performance decentralized network". Without the need for complex layer separation or advanced cryptographic tricks, this model represents a significant advancement in PoW consensus, enabling the processing of large-scale data while preserving the intrinsic simplicity of Bitcoin.

The remainder of this paper is organized as follows. Section 2 reviews the existing literature on blockchain scalability and identifies the fundamental limitations of current consensus mechanisms regarding propagation latency. Section 3 introduces the proposed architecture, detailing the Extended Header, the Compact Block structure with 6-byte compressed IDs, the Mempool Freezing system, and the Context Specific Chain Selection Rules. Section 4 and 5 provide a security and performance analysis, evaluating the collision probability of compressed IDs and the improvements in network throughput and stability. Section 6 presents a discussion on the protocol's extensibility, including block interval reduction and potential economic incentive models for honest declarations. Section 7 concludes the paper by summarizing the key contributions and suggesting future research directions.

2. RELATED WORK

This section categorizes and reviews the previous research conducted over the past decade to resolve blockchain scalability issues into three major categories: data propagation, consensus structures, and off-chain solutions.

2.1. Propagation and Bandwidth Optimization Research

Pioneering research has been conducted to overcome bandwidth constraints and propagation delays, which serve as the primary barriers to blockchain scalability.

BIP 152 (Compact Blocks) [14]: This protocol revolutionized block propagation by transmitting 6-byte short IDs instead of full transaction data, reducing the propagation size by over 90%. While it has become a standard, it possesses a critical limitation: if the mempool states between nodes are inconsistent, additional data requests (getblocktxn) are triggered. This results in unpredictable round-trip latencies, exacerbating propagation delays.

Graphene and Xthin [15]: These approaches utilized Bloom Filters and IBLT to compress propagation data toward its theoretical lower bound by leveraging mempool knowledge. However, the asynchronous nature of mempools often leads to decoding failures and subsequent synchronization overhead. More importantly, these methods fail to escape the inherent serial structure where mining must halt whenever a propagation delay occurs.

Falcon and FIBRE [16]: These solutions attempted to reduce block propagation latency to tens of milliseconds by utilizing fiber-optic networks and relay servers at the physical layer. While they contributed significantly to hardware-level optimization, they entail high costs and centralization risks, making them difficult to apply universally across a global, decentralized node environment.

2.2. Consensus & Parallelization

Innovative consensus algorithms have been proposed to enhance processing speed by refining the conventional serial block production structure.

Bitcoin-NG [10]: This protocol decoupled blocks into "Key Blocks" for leader election and "Microblocks" for transaction records. By allowing a leader node to continuously produce microblocks until the next key block is mined, it successfully increased throughput. While this mitigated the dependency between propagation delay and mining, it introduced new challenges, such as the risk of Denial-of-Service (DoS) attacks on leader nodes and increased verification complexity.

GHOST (Greedy Heaviest Observed Subtree) [11]: This algorithm sought to prevent security degradation caused by propagation delays by including "Uncle Blocks" (orphaned blocks) in the cumulative weight of the chain. Although adopted by networks like Ethereum to enhance security, its impact was limited to accommodating time-lagged blocks to maintain security under high orphan rates rather than maximizing true parallel data availability.

DAG (Directed Acyclic Graph) [17]: Nonlinear structures attempted in projects like IOTA and Fantom sought theoretical infinite parallelism by allowing blocks to reference each other in a mesh-like structure. However, this approach significantly complicates the determination of deterministic transaction order (finality) and double spend validation, often undermining the blockchain's core value of simple and transparent consensus.

2.3. Layer 2 and Data Availability Solutions

A significant body of research focuses on achieving scalability by building upper layers rather than altering the fundamental design of the main ledger.

Payment Channels (Lightning Network) [13]: This approach enables near infinite off-chain transactions by opening dedicated channels between parties, with only the final settlement recorded on-chain. While it has revolutionized micropayment efficiency, it faces operational constraints such as liquidity locking and complex routing requirements.

Sharding [18]: By partitioning the network into multiple "shards," this method allows for distributed transaction processing. Although it significantly increases throughput, it introduces fragmentation of the security model and adds immense complexity to cross-shard communication and defense against single shard attacks.

Rollups (Optimistic & ZK) [19]: Rollups perform computations off-chain and submit only summarized data or zero knowledge proofs to the main chain. While these are the dominant scaling strategies for modern blockchains, they remain inherently limited by the performance of the main chain, as the cost of ensuring Data Availability (DA) is still tethered to the base layer's capacity.

3. PROPOSED SYSTEM DESIGN

This section details the technical specifications of the proposed asynchronous block propagation and verification protocol. The system consists of three core modules: the post declaration mechanism, UTXO based logical isolation, and the consensus enforcement system.

3.1. Data Structure Design

The proposed protocol utilizes three distinct types of data structures, which differ from the standard Bitcoin protocol, to enhance efficient block propagation and verification. These structures form the core components of the protocol's operation.

3.1.1. Block Header

The Block Header is the most critical fixed-length data structure in the proposed protocol and serves as the target for the PoW computation. Furthermore, it acts as a logical linkage between the Compact Block (CB) and the Full Block (FB). The block header of the proposed system strictly inherits the standard 80 bytes structure of Bitcoin while appending 33 bytes for asynchronous finality and tamper proofing, resulting in a total of 113 bytes. These additional fields allow the miner to certify the validity of the $(n - k)$ -th block and enable the receiver to immediately detect any tampering of the Compact Block.

Table 1: Detailed Description of Block Header Fields

Field Name	Size (Bytes)	Description	Remarks
Version	4	Block version number	Inherited from Bitcoin
Prev_Block_Hash	32	Hash of the previous $(n - 1)$ -th block header	Inherited from Bitcoin
Merkle_Root	32	Merkle tree root of all transactions (FB)	Inherited from Bitcoin
Timestamp	4	Block generation time (UNIX Epoch)	Inherited from Bitcoin
Difficulty_Target	4	The difficulty target value for the current n -th block. Inherited from Bitcoin	
Nonce	4	Arbitrary value for PoW computation	Inherited from Bitcoin
$n_k_declaration$	1	Declaration of $(n - k)$ -th block validity (0: False, 1: True)	Key Mechanism
Comp_UTXO_Root	32	Merkle root of the Compressed UTXO ID List	Key Mechanism

Key Features and Role:

PoW Target: The entire 113bytes data, including all the above fields (especially the nonce), is used as input for the SHA-256 algorithm. The resulting hash value must be less than the difficulty target.

Security and Integrity: The Merkle_Root and Comp_UTXO_Root provide strong cryptographic evidence of the Full Block and Compact Block contents, respectively, thereby preventing data tampering.

3.1.2. Compact Block

The Compact Block is assigned the highest priority for network propagation, enabling nodes to decide on immediate mining entry and transaction freezing even before the Full Block is received. The 6-byte UTXO compressed ID list ensures that even for large scale blocks, the transmission volume remains in the range of tens of kilobytes (KB), supporting large scale scalability.

Table 2: Detailed Description of Compact Block Fields

Field Name	Size (Bytes)	Description	Role
BlockHeader	113	As defined in Section 3.1.1	PoW verification and $n_k_declaration$ check
Input_UTXO_Count	Variable (1-9)	The total number of UTXOs consumed as inputs by the transactions within the block	Defines the length of the Compressed ID list

UTXO_Comp_IDs	Variable	6-byte compressed ID list of input UTXOs	Mempool Freezing filter
---------------	----------	--	-------------------------

Key Features and Role:

Immediate Mining Trigger: Enables nodes to commence mining the next block immediately upon receiving the Compact Block, effectively eliminating idle time caused by Full Block propagation.

Logical Isolation: Facilitates instantaneous Mempool Freezing by using the Input_UTXO_Count and compressed ID list, ensuring logical decoupling between adjacent blocks without full data.

3.1.3. Full Block

The Full Block contains the complete data of all transactions to be permanently recorded on the blockchain ledger. It propagates relatively slowly in the background after the compressed block has been disseminated, and nodes use it to perform 'delayed verification'. The Full Block serves as the definitive data entity of the chain and is the subject of verification for post confirmation at the $n + k$ stage. The Full Block proves its adherence to the logical isolation rules by demonstrating that it does not utilize any UTXOs generated after the $n - k$ block, thereby ensuring the stability of the asynchronous structure.

Table 3: Detailed Description of Full Block Fields

Field Name	Size (Bytes)	Description	Role
Block Header	113	The block header structure defined in Table 1 . It shares the same header as the Compact Block.	Cryptographic binding of data and header
Transaction Count	Variable (1-9)	The total number of transactions	Ledger recording and final validity check
Transaction Data	Variable	All actual transaction data within the block	

Key Features and Role:

Basis for Full Verification: Upon receiving this block, a node performs final validation of all transactions, including signatures and UTXO status.

Data Storage: Once verified, the Full Block is stored in the local database, constituting the blockchain ledger.

3.2. Asynchronous Propagation & Mining Process

This protocol independently decouples block propagation speed from mining continuity, ensuring that the entire network's effective hash power operates without idle time.

3.2.1. Dual Propagation & Immediate Mining

Simultaneous Broadcast: A successful miner broadcasts both the $n - th$ Compact Block and Full Block simultaneously. Due to its minimal size, the Compact Block propagates with extreme agility across the network before the Full Block transmission gains momentum.

Preemptive Verification & Mining Entry: Neighboring nodes perform the "Compact Block Validation" (as detailed in Section 3.2.3) within milliseconds of receiving the Compact Block. If valid, the nodes immediately commence mining the $(n + 1) - th$ block, bypassing the need to wait for the arrival of the large $n - th$ Full Block.

Latency Hiding: The physical propagation delay of the Full Block is effectively hidden within the PoW computation interval of the subsequent blocks. This architecture ensures that even with large-scale blocks, the risk of orphan blocks is eliminated, maintaining robust network stability.

3.2.2. Accelerated Convergence via Coupling

A. Coupling Mechanism and Propagation Rules

Universal Propagation: If the $n - th$ Full Block is valid, it is propagated throughout the entire network until all nodes receive it.

Coupling Condition: If the $n - th$ Full Block encounters an $(n + k) - th$ Compact Block with $n_k_declaration = True$ at an intermediate node, they are coupled into a single packet and propagated together. (In most cases, the $n - th$ Full Block will have already saturated the network before the $(n + k) - th$ Compact Block is mined).

B. Impact on Specific Node States

This coupled propagation specifically targets and influences nodes that are currently mining or have already mined the $(n + k) - th$ block with a “False” declaration due to data latency.

Nodes in Progress $n + k$ Mining with “False”: Nodes that were mining the $n + k$ block while assuming the $n - th$ Full Block was missing ($n_k_declaration = False$) will immediately switch their declaration to “True” upon receiving the $n - th$ Full Block. They then restart or pivot their mining process toward the $n + k$ block with the updated “True” status. Furthermore, if they receive a coupled packet (containing both the $n - th$ Full Block and the $n + k$ Compact Block), they immediately transition to the “True chain” and commence mining the $n + k + 1$ block.

Nodes with Completed Blocks (Mining $n + k + j$ with “False”): Nodes that have already succeeded in mining the $n + k$ block with $n_k_declaration = False$ and are currently working on subsequent blocks ($n + k + j$, where $j > 0$) do not perform an immediate chain switch upon receiving the $n - th$ Full Block or the coupled packet. Instead, they record the arrival of the data and detect that a fork has occurred. These nodes will only undergo a reorganization (Reorg) if the “True chain” eventually proves to be the longest chain according to the consensus rules.

3.2.3. Verification Sequence

This section describes the strict step-by-step verification procedures used by nodes to determine the legitimacy of received data, ensuring integrity in high throughput environments. As with the Bitcoin protocol, any data unit in our scheme including transactions, Compact Blocks, and Full Blocks is ignored if it fails verification. Such invalid data is neither propagated further through the network nor retained, resulting in its immediate elimination from the system.

A. Header Validation

First, the node verifies that the block header's PoW meets the current network difficulty target. It also checks that the $n_k_declaration$ field complies with the binary specification (0 or 1).

B. Compact Block Validation

A preliminary verification phase performed on the Compact Block alone, prior to Full Block arrival.

Freeze Duplicate Check: The node checks if the compressed UTXO IDs in the Compact Block overlap with elements already in the local node's Frozen Set (See 3.3.1). If a duplicate is found, it is flagged as a double spend attempt and the Compact Block is rejected immediately.

Merkle Root Comparison: The node constructs a Merkle tree from the received compressed ID list and verifies that the resulting root matches the `Comp_UTXO_Root` in the header.

C. Full Block Validation

A deep verification phase for the definitive data entity.

Transaction Validity: All transaction signatures and script integrity within the block are verified.

Compressed List Consistency: The 6-byte compressed IDs derived from the Full Block's actual transaction inputs must be a 100% match with the Compact Block's list. Any discrepancy is treated as data tampering, and the block is invalidated.

Isolation Rule Check: The node verifies whether the transactions in the $n - th$ Full Block consume UTXOs generated during the logical isolation period ($n - k$ to $n - 1$). The $n - th$ Full Block is strictly prohibited from consuming any UTXOs generated during the logical isolation period (from block $n - k$ to $n - 1$) that have not yet been finalized as “True”. Any block violating this rule is discarded immediately.

3.2.4. Strategic Propagation Delay for Bandwidth Optimization

To further enhance network efficiency and eliminate redundant data transmission during transient forks, this protocol implements a "Strategic Propagation Delay" for Full Blocks.

A. *i*-Confirmation Gated Propagation

The propagation of the $n - th$ Full Block is strictly gated by the cumulative PoW of its corresponding Compact Block chain. A miner or a node shall only commence the universal broadcast of the $n - th$ Full Block after the $n - th$ Compact Block has been included in the longest chain and has achieved i subsequent confirmations (i.e., at the height $n + i$). For general network stability, $i = 2$ is established as the default threshold.

B. Impact on Fork Resolution

Suppression of Orphan Data: During a brief fork at height n , multiple competing Compact Blocks may exist. By withholding the heavy Full Blocks until the Compact Block level conflict is resolved (usually within 1-2 blocks), the network avoids the "Bandwidth Pollution" caused by the simultaneous propagation of multiple, conflicting heavy Full Blocks.

Winner Takes All Efficiency: Once the $(n + i) - th$ Compact Block is mined, it signals a definitive PoW consensus on which $n - th$ block is the legitimate ancestor. Only the Full Block belonging to the "Winning Chain" is then allowed to saturate the network. This ensures that 100% of the network's bandwidth is dedicated to valid, persistent data.

Deterministic Arrival Window: Since the $n - th$ Full Block starts its journey at the $n + i$ mark, it still possesses a $r(= k - i)$ block window (approximately 40 minutes in a 10-minute interval system when $i = 2$ and $k = 6$) to reach all nodes before the final $n + k$ evaluation. This provides a robust safety margin that reconciles the physical limits of data transmission with the logical requirements of consensus. Even if the $(n + k) - th$ Full Block is not received by the time $n + 2k$ is reached, honest nodes will declare $n_k_declaration = \text{False}$ in the $(n + 2k) - th$ block based on their observations. Consequently, the $(n + k) - th$ block is naturally rejected (switched off), allowing the pipeline to proceed to the next block without interruption.

The Nature of the Gated Propagation Mechanism: The i -confirmation gated propagation mechanism is not strictly required for the correctness of the consensus protocol itself. Rather, it serves as a network-level stabilization mechanism that reduces propagation contention and mitigates data flooding risks associated with large Full Blocks.

3.3. UTXO Freezing and Logical Isolation

This protocol applies dynamic freezing control at the mempool level and isolation principles to asynchronously manage data dependencies between blocks.

3.3.1. Definition of the Frozen Set and Atomic Update Logic

This section defines the atomic mechanism that simultaneously handles Asset Isolation (Freezing) for future mining and State Finalization for past blocks, triggered by the arrival of a Compact Block.

A. Definition of the Frozen Set

The Frozen Set is defined as a collection of compressed UTXO IDs that are restricted from being used as inputs during the k -block isolation period. It functions as a dynamic input filter to ensure transaction consistency.

B. Atomic Swap of IDs (Simultaneous Register & Release)

Upon receiving the $n - th$ Compact Block, the node executes the following two operations as a single atomic transaction to maintain optimal mempool efficiency:

New Registration (Register n): The UTXO_Comp_IDs from the $n - th$ Compact Block are immediately added to the Frozen Set. This prevents the $(n + 1) - th$ block from including any transactions that conflict with the $n - th$ block.

Immediate Release (Release $n - k$): Concurrently, the node removes the UTXO_Comp_IDs associated with the $(n - k) - th$ block from the Frozen Set. Since the $n - th$ header already contains the fate of the $(n - k) - th$ block via the $n_k_declaration$, there is no technical necessity to maintain these IDs in the isolation list.

C. Candidate Block Construction Principles

When constructing the $(n + 1) - th$ candidate block, mining nodes apply the following filtering rule to the mempool.

Rule: "Any transaction that contains a `UTXO_Comp_ID` belonging to the Frozen Set as one of its inputs must be excluded from Selection."

D. Acceptance of Over Freezing

Due to the nature of 6-byte (48-bit) compressed IDs, accidental hash collisions (as explained by the Birthday Paradox) may cause unrelated transactions to be frozen. However, this protocol adheres to the strict security principle of "allowing false positives (over freezing) while prohibiting false negatives (missed freezing)". Since an accidentally frozen transaction is simply released after k blocks to be included in a later block, this conservative freezing is accepted as a necessary trade-off to enhance overall system integrity and stability.

3.3.2. Consistency and Release of Freeze

Concurrent with the release of IDs from the Frozen Set, the node finalizes the global ledger (UTXO Set) based on the $n_k_declaration$ value found in the $n - th$ Compact Block.

A. Case 1: $n_k_declaration=True$ (Finalization)

Ledger Update: The actual UTXOs associated with the $(n - k) - th$ block are permanently marked as "Spent" in the ledger.

Mempool Cleanup: Transactions that were successfully included and finalized are purged from the Mempool to optimize memory resources.

B. Case 2: $n_k_declaration=False$ (Invalidation & Recovery)

Asset Restoration: As the $(n - k) - th$ block is deemed invalid, the transactions associated with it are restored to a "spendable" status within the Mempool.

Isolation Integrity: Due to the logical isolation, blocks from $(n - k + 1)$ to $(n - 1)$ remain unaffected by this invalidation. The restored assets are eligible for inclusion starting from the $(n + 1) - th$ candidate block.

3.3.3. Architectural Isolation Rules for Logical Decoupling

This section defines the core isolation rules necessary to physically break the data dependencies and logically decouple blocks, ensuring integrity within the asynchronous mining environment.

No Chaining Rule (Prohibition of New Asset Use): Any new UTXOs generated within the $n - th$ block (including change outputs) are in an unconfirmed state until finalized at block $n + k$. Therefore, these UTXOs absolutely cannot be used as inputs in any transactions within blocks $n + 1$ through $n + k$. This extends Bitcoin's coinbase maturity concept (100-block wait) to general transactions for a duration of k blocks.

Independent Validity Guarantee: Adherence to this rule ensures that even if the $n - th$ block is invalidated ($n_k_declaration=False$), all subsequent blocks (from $n + 1$ onwards) maintain their independent validity. They are guaranteed to have been created only using UTXOs already finalized before the $n - k$ point. This prevents cascading invalidation and maximizes the system's resilience.

3.4. Fork Convergence Algorithm

To rapidly integrate temporary branches in an asynchronous environment, this protocol inherits Bitcoin's Longest Chain Rule but integrates a unique convergence mechanism combined with "Data Availability Verification."

3.4.1. Pipelined Consensus Mechanism

This section formally defines the pipelined consensus mechanism, which is described in Section 3.1, 3.2 and 3.3. This mechanism overcomes the sequential constraints of legacy systems by decoupling mining from finalization through a k -stage temporal phase shift.

A. Notations and Definitions

B_n, CB_n : $n - th$ Full Block and Compact Block.

S_n : Ledger state finalized up to block n . In Bitcoin, this refers to the finalized UTXO set.

TX_n : Mempool state.

C_n : Filter state (Frozen Set) updated via CB_n .

$NewTX_n$: Set of newly incoming transactions.

$CB_n.n_k_declaration$: Validity declaration for block $(n - k)$ in CB_n header.

$t_{start}(B_n), t_{end}(B_n)$: Start and end times of B_n mining.

$Pow(B_n)$: Pure computation time for mining $t_{end}(B_n) - t_{start}(B_n)$.

$T(B_n), T(CB_n)$: Propagation times for B_n and CB_n to reach >90% of nodes.

F, G, H : Functions for state transition, mempool update, and filter update.

B. Bitcoin's Sequential Consensus Model ($k = 0$)

In Bitcoin, consensus proceeds atomically and sequentially on a single timeline. The transition is as follows:

Transitions:

1. Mining: $B_n = BlockGenerate(S_{n-1}, TX_{n-1})$

A miner generates block B_n by assembling transactions from the current mempool TX_{n-1} atop the finalized historical ledger state S_{n-1} . This process cannot commence until the prior state S_{n-1} has been fully received and validated.

2. Finalization: $S_n = F(S_{n-1}, B_n)$

Upon reception of B_n (provided it is valid), a node instantly updates its local ledger state from S_{n-1} to S_n . Crucially, the mining of the subsequent block cannot begin until this ledger update is complete.

3. Mempool Update: $TX_n = G(TX_{n-1}, B_n, NewTX_n)$

Transactions included in B_n are purged from the mempool, and newly arrived transactions $NewTX_n$ are added to the waiting pool.

Temporal Constraint: $t_{start}(B_{n+1}) = t_{end}(B_n) + T(B_n)$

The start of mining for B_{n+1} is directly dependent on the full propagation of B_n , making the system highly sensitive to propagation latency. The critical vulnerability of Bitcoin lies in the sequential dependency of its mining initiation point. Consequently, the $(n + 1) - th$ block is forced to wait for the duration of $T(B_n)$, the time required for the $n - th$ block to propagate fully across the network. This bottleneck is the direct cause of the sharp decline in total network throughput as block sizes increase and $T(B_n)$ lengthens. While Bitcoin's Compact Block Relay (BIP 152) attempts to mitigate this by reducing $T(B_n)$, it fundamentally fails to overcome the limitations inherent in this sequential architecture.

C. Proposed Pipelined Consensus Model ($k = 6$)

The proposed model separates mining initiation from ledger finalization using a k -stage pipeline.

Transitions:

1. Mining: $(B_n, CB_n) = BlockGenerate(S_{n-k-1}, TX_{n-1}, C_{n-1}, CB_{n-1})$

The $n - th$ block is mined based on the finalized state S_{n-k-1} from $(n - k - 1) - th$ stages prior, while simultaneously integrating the latest mempool TX_{n-1} and the dynamic filter C_{n-1} to ensure transaction consistency.

2. Filter Update: $C_n = H(C_{n-1}, CB_n)$

Upon reception, the Compact Block CB_n immediately updates the Frozen Set state C_n , which serves as the filter for the subsequent block $n + 1$. This process is completed within the minimal duration of $T(CB_n)$.

3. Finalization: $S_{n-k} = F(S_{n-k-1}, B_{n-k}, CB_n.n_k_declaration)$

At the moment the $n - th$ block emerges, the ledger state transitions from S_{n-k-1} to S_{n-k} according to the validity declaration $n_k_declaration$ assigned to the $(n - k) - th$ block.

4. Mempool Update: $TX_n = G(TX_{n-1}, B_{n-k}, CB_n.n_k_declaration, NewTX_n)$

Based on the data of the finalized block B_{n-k} , the mempool is updated by purging($CB_n.n_k_declaration=True$) or restoring($CB_n.n_k_declaration=False$) consumed transactions and accepting newly incoming transactions $NewTX_n$.

A conventional pipeline operation follows the timing rule:

$$t_{start}(B_{n+1}) = \max\{t_{end}(B_n) + T(CB_n), t_{end}(B_{n-k+1}) + T(B_{n-k+1})\}$$

When considering the i -confirmation-gated propagation, the aforementioned equation is modified as follows:

$$t_{start}(B_{n+1}) = \max\{t_{end}(B_n) + T(CB_n), t_{end}(B_{n-k+1+i}) + T(B_{n-k+1})\}$$

In this model, if the right-hand term representing the propagation of the heavy Full Block $T(B)$ exceeds the left-hand term, a pipeline stall occurs, forcing the system to wait and wasting network resources.

In the proposed system, the arrival status of the Full Block is decoupled from the mining initiation trigger. Consequently, the start time is always pegged to the left-hand term:

$$t_{start}(B_{n+1}) = t_{end}(B_n) + T(CB_n)$$

The node does not treat the reception of B_{n-k+1} as a prerequisite for mining. Instead, it simply selects the declaration value “True” if the data has arrived, or “False” if it has not and continues the mining process without interruption.

t_{start} is determined exclusively by the Compact Block CB_n , which has a minimal and consistent propagation delay.

Any delay in $T(B_{n-k+1})$ affects only the 1-bit declaration value $n_k_declaration$. In essence, data latency becomes a matter of “what to declare” rather than “when to start.”

If the propagation delay $T(B_{n-k+1})$ exceeds the physical threshold of the k -window, the system opts for asynchronous forking instead of a pipeline stall. This prioritizes the Liveness of the network, ensuring that consensus momentum is never sacrificed for data synchronicity.

As a result, the proposed mechanism ensures that the consensual pulse of the network remains constant even when data moves slower than the mining speed. Since $T(CB) \ll T(B)$ on average, t_{start} is effectively pegged to the propagation speed of the Compact Block.

Note. If the block size, mining interval, k and i are configured appropriately (e.g., block size of 10MB, mining interval of 600s, $k = 6$ and $i = 2$), $t_{end}(B_n) + T(CB_n) > t_{end}(B_{n-k+1+i}) + T(B_{n-k+1})$ holds with high probability (See Section 5.2). Even if a fork occurs due to the delay of B_{n-k+1} , the system operates on the premise that any forks will be resolved through the longest-chain rule and Proof of Work competition within the sufficient temporal buffer provided by k . The fork convergence algorithm is described below.

3.4.2. Context-Specific Chain Selection Rules

When a node discovers a longer chain (B) of length b_len than its current one (A) of length a_len , it follows a sophisticated algorithm to prevent ghost chain attacks by integrating data availability checks into the longest chain rule. Assuming that Group A and Group B chains are consistent up to the $n - th$ block (the fork point), the relative progress after the fork is represented as $a_z = a_len - n$ for Group A and $b_z = b_len - n$ for Group B. Let $m = b_len - a_len$.

A. Threshold Convergence ($m = 1$ and $a_z = 0$)

At the moment a fork starts, the node performs an “objective verification based on subjective observation.” It compares the $n_k_declaration$ in the $b_len - th$ Compact Block of chain B with its local data status. The protocol strictly defines the verification timing to ensure consistency between subjective observation and objective consensus. At the exact moment a node receives the $b_len - th$ Compact Block, it performs the following steps:

Local Status Evaluation: The node checks the status of the $b_len - th$ Compact Block of chain B with its local data status.

If the $(b_len - k) - th$ Full Block is present and valid, the local status is set to “True”.
Otherwise, the local status is set to “False”.

Decision Logic: The node compares this local status with the $n_k_declaration$ field in the $b_len - th$ Compact Block header.

If the local status matches the declaration, the $b_len - th$ Compact Block is accepted.
Otherwise, the block is rejected immediately.

The switch only occurs if the declaration matches the actual data availability. This prevents confusion caused by “False” declarations without data.

B. Deterministic Convergence ($m = 1, a_z > 0$ or $m > 1$)

If a clear length difference exists, the longest chain rule is prioritized under conditional acceptance.

False-declared blocks: The node accepts the B chain's "False" evaluation to maintain consistency, even if it possesses the data.

True-declared blocks: The node must acquire and verify all Full Block data for blocks declared as "True" before finalizing the switch. No matter how long a chain is, it is never accepted without the underlying data.

Nodes only switch to a longer chain after verifying all data for blocks declared as "True", preventing ghost chain attacks. In the event of a prolonged network partition, the fork resolution process via the longest chain rule necessitates significant overhead, as nodes must download and verify a vast number of accumulated blocks. This characteristic is inherent to any Nakamoto-consensus-based protocol, including Bitcoin.

3.4.3. Resolving Information Asymmetry via Coupling Mechanism

Bottleneck Elimination and Sync Efficiency: Coupling Mechanism (as detailed in Section 3.2.2.) physically eliminates the standby time required for "data re-requests and downloads" during a fork switch. Among honest nodes, data and consensus proof are organically shared via Coupling Mechanism. Consequently, upon receiving a new block, consensus convergence occurs instantly through Compact Block verification alone, eliminating additional data request cycles. This terminates forks significantly faster than the original Bitcoin protocol.

Suppression of Anomalous Attacks: Even if an attacker attempts to deceive the network with a "longer chain" by leveraging high hash power while withholding the actual data (Full Block), the "Mandatory Data Verification" rule ensures that nodes without the data will not accept the attacking chain. As a result, the malicious chain is rejected by the majority of honest nodes and becomes isolated within a "quagmire of propagation delay."

This algorithm provides incentives for honest nodes through "fast propagation" while imposing physical barriers propagation delays and resource exhaustion on malicious actors. This serves as an economic and technical deterrent, forcing attackers to incur significantly higher costs to compete with honest participants.

3.5. System Initialization and Bootstrapping

The proposed asynchronous block propagation protocol's core mechanisms, the n_k declaration and the UTXO Frozen Set, operate on the premise of a minimum k -length block history. This section details the specific operating procedures for the initial state transition in two main scenarios: bootstrapping a new network from the genesis block and transitioning an existing Bitcoin network to the proposed protocol.

3.5.1. Bootstrapping from the Genesis Block

For a newly initiated blockchain where the Activation Height is set to block 1, the processing of the n_k declaration field until k blocks have been generated is defined as follows:

A. Initial k Blocks (Block Heights 1 to k):

The genesis block (height 0) is a special case and is considered to be without the n_k declaration field, or before the protocol is active.

Block headers from height 1 to k include the n_k declaration field, but it is always declared as "False" (0) as no valid $n - k$ block exists at that time.

During this period, the network follows the traditional Bitcoin serial verification model with specific constraints to ensure data integrity and prevent immediate forks:

Unique Compressed IDs: The 6-byte compressed hash values (UTXO_Comp_IDs) within each of the initial blocks (1 through k) must not overlap within the same block to prevent local ambiguity.

Strict Isolation: The "No Chaining Rule" is strictly enforced: an output (new UTXO) generated by block i must not be used as an input in any transaction within blocks $i + 1$ to k . This prevents dependency chains before the isolation mechanism is formally active.

Immediate Full Block Propagation: The Strategic Propagation Delay (Section 3.2.4) is temporarily bypassed. For the initial blocks 1 through k , the Full Block is broadcast immediately upon mining to ensure rapid data availability across the network, given that the i -confirmation gate is not yet functional.

B. Transition Point ($k + 1$ Block):

When the $k + 1$ block is generated, its header's $n_k_declaration$ refers to the validity of the first block (height 1). From this point onwards, all rules of the proposed asynchronous protocol are fully activated.

It is assumed that the typical propagation delay of a Full Block allows honest nodes to receive and verify block 1 data within the time it takes to mine k subsequent blocks (approximately 40 minutes for $k = 6$ in a 10-minute interval system). Honest nodes will therefore validate and set the $n_k_declaration$ to “True” (1).

3.5.2. Transitioning from an Existing Bitcoin Network

To introduce the proposed protocol into a currently operating Bitcoin network, a hard fork mechanism is employed at a predetermined block height.

Activation Mechanism:

Network participants agree to activate the new protocol rules upon reaching a specific, agreed-upon block height (Activation Height).

Before reaching Activation Height: The network operates according to the existing Bitcoin protocol.

After reaching Activation Height: Blocks generated from this point forward must use the new, extended 113-byte header structure and comply with the $n_k_declaration$ and UTXO Freezing rules.

For the purpose of the initial transition, the k blocks immediately preceding the Activation Height are assumed to be a valid, finalized history. The $n_k_declaration$ field for the first k new blocks will default to “True” (1), and the isolation rules defined in 3.5.1 will apply during this initial phase.

4. SECURITY ANALYSIS

This section verifies the security resilience and engineering efficiency of the proposed protocol. The analysis is based on the core principle: "Allow false negatives (misunderstandings) but architecturally block false positives (fraud)."

4.1. Fork Convergence and Resolution Analysis

The proposed protocol achieves a superior level of network stability compared to Bitcoin by decoupling the logical consensus process from physical data synchronization. This section demonstrates how our system resolves forks faster, cleaner, and with significantly less computational overhead than the traditional Bitcoin model.

4.1.1. Fork Convergence Equivalence and Structural Efficiency

The proposed mechanism does not fundamentally complicate fork dynamics compared to Bitcoin. Rather than redefining the convergence rule itself, the protocol primarily regulates the initial condition under which a fork emerges. The core convergence principle remains unchanged (See 3.4.2).

We distinguish two types of fork events in the proposed protocol:

- ♦ A-type fork: a fork induced by Compact Block propagation delay.
- ♦ B-type fork: a fork induced by inconsistency in the $n_k_declaration$ field.

We show that, in both cases, the post-divergence convergence process reduces to the same stochastic model as in Nakamoto consensus.

A. A-Type Fork (Compact Block Propagation Delay)

An A-type fork arises solely from bounded network latency in the dissemination of Compact Blocks. In this case, two competing Compact Blocks at the same height may be received by different subsets of miners, resulting in a temporary divergence.

However, this scenario is structurally identical to the classical fork condition in Bitcoin. The only difference lies in the payload format (Compact Block rather than Full Block), not in the consensus logic governing branch selection.

Once the divergence occurs, miners extend the branch they first observed, and subsequent convergence is determined exclusively by relative hash power under the longest-chain rule. No additional protocol-level state or validation rule influences branch selection in this case.

Therefore, the post-divergence evolution of an A-type fork reduces exactly to the stochastic longest-chain competition of Nakamoto consensus. Under the standard assumptions of honest majority and bounded network delay, the probabilistic convergence guarantees are unchanged.

In short, A-type forks are behaviorally equivalent to Bitcoin forks and require no additional security analysis beyond that already established for Nakamoto consensus.

B. B-Type Fork (n_k declaration Inconsistency)

A B-type fork arises when competing blocks at height $n + k$ contain inconsistent n_k declarations under the protocol's deterministic validation rule. Such inconsistency creates a divergence at the declaration boundary.

This work does not alter Bitcoin's longest chain based probabilistic convergence algorithm. Instead, it introduces structured control over the moment of fork initiation through the timing and verification constraints associated with the n_k declaration field.

While the n_k declaration rule serves as a validity filter at the fork boundary, it leaves the longest-chain extension rule used by honest nodes unchanged.

After the initial divergence point, the system reduces to the same stochastic convergence model as Nakamoto consensus.

In particular, the commonly stated intuition that “a length difference of one block leads to immediate convergence” remains valid only under the following assumptions:

- ♦ The longer chain is valid under the protocol rules.
- ♦ All Full Blocks declared as “True” are made available and verified.
- ♦ An honest majority (i.e., >50% of total hash power) ensures positive drift toward the honest chain.

These assumptions are identical to those required by Bitcoin's security model. Therefore, under the same honest majority and bounded network delay model as assumed in Bitcoin, the proposed protocol preserves the probabilistic fork convergence properties of Bitcoin, while modifying only the control logic governing the initial fork event. This relationship is formally established in the following theorem.

Theorem 1 (Conditional Probabilistic Isomorphism of Fork Convergence Dynamics)

Consider a blockchain protocol in which:

- ♦ Nodes that have received block B_n set n_k declaration = true (Group A),
- ♦ Nodes that have not received block B_n set n_k declaration = false (Group B).

Let a_t and b_t denote the respective chain lengths maintained by the two groups.

Assume that no fork exists up to height $n - k + 1$, and that a fork arises at height $n + k$.

Then, although the initial fork configuration differs from that of Bitcoin due to partial data availability in the interval $[n, n + k - 1]$, the subsequent fork convergence process is conditionally probabilistically isomorphic to the longest-chain fork resolution process of Bitcoin, once the system enters the deterministic convergence regime.

Proof.

1. Exhaustive Case Decomposition (Threshold Convergence Regime)

At height $n + k$, the fork is initiated under conditions corresponding to Section 3.4.2.A (Threshold Convergence, $m = 1, a_z = 0$).

Unlike Bitcoin, where all blocks up to height $n + k - 1$ are globally shared, the proposed protocol allows for partial data availability in the interval $[n, n + k - 1]$. Specifically:

- ♦ Full Blocks $B_n, \dots, B_{n+k-1}$ may not be globally available,
- ♦ However, Compact Blocks $CB_n, \dots, CB_{n+k-1}$ are assumed to be consistently propagated.

As a result, the fork at height $n + k$ is not solely determined by propagation delay, but also by data availability asymmetry, which directly affects the n_k declaration field. Thus, the initial fork configuration differs from that of Bitcoin and must

be treated as a distinct threshold regime. Nevertheless, this establishes the same structural branching as in Bitcoin, albeit with an augmented state condition. This can be further elaborated as follows.

At height $n + k$, exactly two initiating events are possible:

Case A: Group A mines first

- ♦ If block n propagates to Group B before B produces a competing block at height $n + k$, the fork resolves deterministically in favor of A.
- ♦ Otherwise, both groups hold chains of equal height:

$$(a_{len}, b_{len}) = (n + k, n + k),$$

entering a balanced competitive state.

Case B: Group B mines first

- ♦ The system temporarily enters an unbalanced state:

$$(a_{len}, b_{len}) = (n + k - 1, n + k).$$

- ♦ If B mines again before A reaches height $n + k$, convergence occurs deterministically in favor of B.
- ♦ Otherwise, equality is restored:

$$(a_{len}, b_{len}) = (n + k, n + k).$$

Therefore, all non-deterministic outcomes reduce to the symmetric configuration:

$$a_{len} = b_{len}.$$

No additional state variables influence subsequent evolution ($m = 1, a_z > 0$).

2. Transition to Deterministic Convergence ($m = 1, a_z > 0$)

Once a chain obtains a length advantage of at least one block beyond the fork point, the system enters the regime described in Section 3.4.2.B (Deterministic Convergence).

In this regime:

- ♦ Chain selection follows the longest-chain rule under conditional validation,
- ♦ Blocks declared as “True” require data verification,
- ♦ Blocks declared as “False” are accepted without requiring full data.

Crucially, in this regime, the effect of earlier data availability asymmetry becomes negligible:

- ♦ The chain selection decision depends only on relative chain length and validity conditions,
- ♦ The internal differences in data availability over $[n, n + k - 1]$ no longer influence the stochastic evolution of the fork.

Therefore, the system reduces to a one-dimensional stochastic process governed solely by chain length difference.

3. State Space Reduction

Define the state variable:

$$\Delta_t := a_t - b_t.$$

where:

a_t is the length of Group A’s chain extension beyond block $n - 1$,

b_t is the length of Group B’s chain extension beyond block $n - 1$.

In the balanced state, $\Delta_0 = 0$.

Each subsequent block discovery updates:

$$\Delta_{t+1} = \begin{cases} \Delta_t + 1 & \text{with probability } \alpha, \\ \Delta_t - 1 & \text{with probability } 1 - \alpha, \end{cases}$$

where α is the hash power fraction of Group A.

Fork resolution occurs at the stopping time

$$\tau = \inf\{t: |\Delta_t| = 1\}.$$

This defines a one-dimensional Markov chain with absorbing boundaries at ± 1 and the state space is effectively reduced to that of Bitcoin, as:

- ♦ Data availability no longer introduces additional independent state variables,
- ♦ Transition probabilities depend only on hash power distribution.

4. Long-Range Forks ($m > 1$)

In the case of prolonged network partition, where $m > 1$, the system remains within the deterministic convergence regime.

Even in this case:

- ♦ Any chain that achieves a length advantage is selected,
- ♦ Data availability asymmetry does not alter the selection rule,
- ♦ The convergence process remains governed by the longest-chain principle.

Thus, the same stochastic process applies.

5. Isomorphism Argument

While the initial fork configuration differs due to partial data availability, the system enters a regime in which the additional state variables become irrelevant.

From that point onward:

- ♦ The state variable is identical to Bitcoin,
- ♦ Transition probabilities are identical,
- ♦ Absorbing conditions are identical.

Therefore, the fork convergence dynamics are conditionally probabilistically isomorphic to those of Bitcoin. $\square$

Moreover, in certain respects, the proposed mechanism exhibits structural efficiency advantages over Bitcoin. In particular, blocks explicitly declared as “False” do not require Full Block download and verification for chain selection, thereby reducing redundant data transfer and validation overhead. By contrast, in Bitcoin, nodes must download and fully validate all competing blocks before safely switching to a longer chain. This asymmetry reduces unnecessary bandwidth consumption during transient fork conditions while preserving equivalent security assumptions.

4.1.2. Structural Advantages in Fork Resolution

A. Agility of Convergence: Compact Block Driven Resolution

In Bitcoin, the convergence of a fork is strictly bound to the propagation speed of Full Blocks, which often exceed several megabytes. This makes fork resolution sensitive to full-block propagation latency. In contrast, our protocol utilizes Compact Blocks consisting only of headers and compressed transaction IDs as the primary vehicle for fork resolution.

Comparison: While a several MB Bitcoin block may experience non-negligible propagation delay across a globally distributed network, our KB sized Compact Blocks propagate significantly faster due to their reduced size.

Result: Fork resolution is effectively determined by the propagation of Compact Blocks carrying valid PoW headers. Given their significantly smaller size, the network identifies the “winning chain” within 1-2 blocks (PoW statistical average) almost

without friction. This ensures that the fork state is ephemeral, whereas Bitcoin's forks are prolonged by the mass of the data being moved.

B. Network Hygiene: PoW Gated Full Block Propagation

Bitcoin's forks are "polluted" because conflicting Full Blocks from all competing chains are broadcast simultaneously, wasting global bandwidth on data that will eventually be discarded (orphan blocks). Our protocol implements the Strategic Propagation Delay (Section 3.2.4), which mandates that an n -th Full Block is only broadcast after its $(n + i)$ -th Compact Block confirmation (typically $i = 2$).

Comparison: Bitcoin nodes may temporarily store competing branches until convergence. In our system, fork competition is primarily resolved at the Compact Block layer. In most cases, the dominant chain is statistically established before Full Block propagation.

Result: Only the Full Block belonging to the winning chain ever saturates the network. This "winner-takes-all" propagation model significantly reduces bandwidth overhead associated with losing branches. Consequently, this design reduces unnecessary full-block propagation in transient fork scenarios, at the cost of introducing a bounded confirmation delay.

C. Computational Simplicity: On/Off Atomic Switching via Pre-verified Data Sharing

In Bitcoin, a reorganization (Reorg) is a heavy "Rollback and Replay" process. Bitcoin reorganizations require rolling back UTXO changes and applying an alternative branch. In contrast, our protocol simplifies this into a near-instantaneous Atomic Switching operation.

Pre-verified Shared Buffer: Consider a fork at height $n + k$ where Group A miners declare $n - th$ block as "True" while Group B declares it "False." While their $n + k$ blocks contain different transactions, the preceding blocks from $n + 1$ to $n + k - 1$ remain identical and are already locally verified by all nodes due to the asynchronous consensus on Compact Blocks.

Logical vs. Computational Reorg: When the fork is resolved within the k -block window (e.g., at $n + 2k$), the "losing" nodes do not need to re-verify or re-execute the transactions of the winning chain. Since the full-block data for the intervening blocks ($n + 1$ to $n + k - 1$) was already shared and validated in the background, the node only needs to perform a structural On/Off switch: activating the winning block's status and deactivating the loser's.

Result: This eliminates the need for re-calculating signatures or script executions during a Reorg. The computational cost is reduced from Bitcoin's $O(N \times T)$ where N is the fork depth and T is the transaction count to a near-constant $O(1)$ atomic database update, ensuring that even massive Reorgs do not cause CPU spikes or network stalls.

4.1.3. Effect of i -Confirmation Gated Propagation on Fork Convergence

A. Independence of Gated Propagation from Direct Fork Resolution

The i -confirmation gated propagation mechanism (A node delays propagation until confirmation $depth \geq i$.) does not directly influence fork convergence in the proposed consensus protocol. The primary reason is that fork resolution is determined by the ordering of Compact Blocks rather than by the propagation timing of the corresponding Full Blocks.

First, fork convergence is governed by the most recently mined Compact Blocks that define the canonical chain ordering.

Since the Compact Blocks are propagated independently of the associated Full Block data, the immediate convergence behavior of forks is largely unaffected by whether the corresponding Full Block has already propagated through the network.

Second, the final validation decision affecting consensus is determined by the Full Block that was mined k blocks earlier.

Because this validation event concerns blocks that are already sufficiently deep in the chain, it is generally independent of the propagation timing of the most recently mined Full Block.

Nevertheless, the i -confirmation gated propagation mechanism may still exert an indirect influence on the system. By design, it delays the network-wide propagation of newly mined Full Blocks by approximately i block intervals. Although this delay does not affect the immediate fork-resolution process, the propagated Full Block will eventually participate in the validation decision once an additional k blocks have been mined. In that sense, the delayed propagation may indirectly influence the later validation phase of the protocol.

Therefore, the i -confirmation gated propagation mechanism should be interpreted primarily as a network-level stabilization mechanism rather than as a direct component of the fork-convergence rule.

B. Interaction Between i -Confirmation Gated Propagation and Universal Propagation

The effectiveness of the proposed i -confirmation gated propagation mechanism becomes more evident when it operates together with the Universal Propagation principle described in Section 3.2.2. Consider the following scenario involving a temporary network disruption.

Assume that when the $n + k - th$ block is mined, the Full Block F_n has propagated to only about 55% of the nodes due to a severe network disturbance. Let this subset of nodes be denoted as group A, while the remaining nodes that have not yet received F_n form group B.

Because nodes in group A have already received the Full Block F_n , they will mine blocks with $n_k_declaration = true$.

In contrast, nodes in group B will mine blocks with $n_k_declaration = false$. As a result, the network temporarily splits into two competing branches.

During this period, the i -confirmation gated propagation mechanism prevents newly mined Full Blocks from being immediately disseminated across the network. Consequently, neither group can quickly propagate their Full Blocks to the other partition, and the fork evolution is primarily determined by the sequence of Compact Blocks. Now consider the moment when the $n + k + z - th$ block ($z \geq 1$) is mined.

If group A mines this block first, the Universal Propagation property implies that there is a significant probability that the Full Block F_n has already reached a substantial portion of nodes in group B during the intervening time. In this case, the effective dissemination window for the Full Block expands from the conservative $k - i$ interval assumed in the security analysis to approximately $k - i + z$ blocks. Once this extended window reaches ($z = i$) or exceeds ($z > i$), nodes across the network are likely to converge toward the chain containing the valid declaration.

A similar situation may arise even if group B mines the $n + k + z - th$ block first. Because Universal Propagation continuously spreads previously mined blocks across the network, the Full Block F_n may already have reached many nodes in group B before the fork is resolved. Of course, in cases where the entire network converges toward the branch maintained by group B, the dissemination of F_n holds no significance for the consensus outcome.

Consequently, the $k - i$ dissemination window used in the security analysis can be interpreted as a conservative lower bound.

In practice, due to Universal Propagation and the delayed dissemination introduced by i -confirmation gating, the effective propagation window may extend to approximately $k - i + z$ blocks, where $z \geq i$. This observation suggests that the proposed mechanism may provide stronger practical robustness than what is implied by the conservative analytical bound (See Section 4.3.2), as the continued dissemination of previously mined blocks can dynamically extend the effective propagation window. As a result, the proposed protocol exhibits an adaptive propagation tolerance, where the minimum dissemination window required for security is bounded by $k - i$, while the effective propagation window may become larger in practice due to the continued dissemination of previously mined blocks.

4.2. Mitigation of Attack Vectors and Resilience Analysis

This section analyzes the system's resilience against malicious attempts to disrupt the network by exploiting the asynchronous nature of data propagation and declaration.

4.2.1. Comparative Resilience Against Bitcoin-Specific Attack Vectors

This section evaluates how the proposed protocol mitigates traditional Bitcoin attack vectors [20-24] by minimizing the opportunities for propagation latency exploitation.

A. Mitigating Idle Time Exploitation

In the standard Bitcoin protocol, attackers benefit significantly from the idle time (or stale block rate) of honest nodes caused by the propagation delay of large Full Blocks. During the interval where honest nodes are synchronizing a new block, their effective hash power is temporarily neutralized, giving the attacker a relative competitive advantage.

Proposed System Superiority: Since fork resolution and chain extension are driven by Compact Blocks which propagate at near instantaneous speeds the network effectively eliminates propagation induced idle time. Honest nodes operate at 100% efficiency immediately upon receiving a Compact Block. Consequently, any attack strategy that relies on the "stale rate" (such as Selfish Mining or Block Withholding) is rendered ineffective, as the "propagation margin" used to outpace the honest majority has been removed. By transforming the consensus race into a contest of pure hash power, the proposed system establishes a security posture that is inherently more robust than the original Bitcoin architecture.

B. Hardening the 51% Attack Barrier

Bitcoin Context: A Bitcoin network can be compromised with slightly less than 51% of raw hash power if the attacker exploits high network latency to increase the honest nodes' orphan rate.

Proposed System Context: By reducing the propagation delay to nearly zero, the protocol forces an attacker to compete against the absolute, 100% hash power of the honest majority. This raises the practical bar for a 51% attack to its theoretical maximum, making the network far more resilient than Bitcoin under similar hardware conditions. Of course, like any other PoW-based consensus, the proposed protocol is not immune to a 51% attack.

While this protocol successfully mitigates traditional Bitcoin based attacks, the introduction of the k -window and n_k declaration fields may create a new, unique attack surface. Future research is required to analyze potential attacks such as "Declaration Manipulation" that are non-existent in Bitcoin. However, the following analysis suggests that any such attempt would face the same economic and physical deterrents (namely, the forfeiture of rewards and the barrier of data transmission) that protect the core consensus.

4.2.2. Resilience Against Selfish Mining: A Formal Analysis

By structurally decoupling PoW consensus competition from Full Block data propagation, the proposed protocol ensures that the profitability of selfish mining is not augmented relative to the traditional Nakamoto Consensus.

In classical Nakamoto style blockchains, an adversary can obtain a strategic advantage by withholding a newly mined Full Block. During the withholding period, honest miners continue to extend a stale tip, thereby wasting hash power and enabling the attacker to increase the probability of achieving a private lead. The effectiveness of such a strategy depends critically on propagation delay and the associated propagation advantage parameter (γ), which governs tie-breaking outcomes during fork races.

Let $\alpha \in (0,1)$ denote the adversary's relative hash power and $\gamma \in [0,1]$ the probability that honest miners adopt the adversary's block in a race condition. The expected revenue of selfish mining can be expressed [23] as a function

$$R_{SM}(\alpha, \gamma) = \frac{\alpha(1 - \alpha)^2(4\alpha + \gamma(1 - 2\alpha)) - \alpha^3}{1 - \alpha(1 + (2 - \alpha)\gamma)}$$

and selfish mining becomes profitable if

$$R_{SM}(\alpha, \gamma) > \alpha.$$

Thus, profitability is fully determined by the pair (α, γ) .

A. Decoupling Consensus from Payload Propagation

In the proposed design, consensus competition is determined exclusively at the Compact Block layer. A Compact Block contains only minimal metadata (e.g., block headers and transaction input identifiers), and its propagation delay is therefore dominated by network latency rather than payload size. Consequently, the additional propagation asymmetry attributable to Full Block transmission is substantially reduced.

Importantly, the timing of Full Block delivery does not influence the establishment of PoW consensus priority. Broadcasting a Compact Block while withholding the corresponding Full Block does not confer a private lead, because honest nodes immediately commence mining upon receipt of the Compact Block. Moreover, if the Full Block is not delivered within the prescribed k -window, the block is deterministically declared invalid ("False") at height $n + k$, resulting in complete forfeiture of the associated block reward. Thus, Full Block withholding cannot increase the adversary's expected revenue.

Accordingly, the expected payoff of selfish mining under the proposed protocol remains governed by the classical parameters α and γ , without introducing any additional gain derived from payload-level manipulation. In other words, the expected payoff structure of selfish mining remains bounded by the same α - γ conditions established in Nakamoto Consensus.

Under the assumption that Compact Block propagation is not slower, nor more asymmetric, than Full Block propagation in the Bitcoin network (See Section 4.3.2), the proposed protocol does not reduce the threshold condition for profitable selfish mining.

B. Granular Analysis of Lead-Reduction Disclosure ($\Delta \rightarrow \Delta - 1$)

We now analyze the canonical selfish mining maneuver at finer granularity.

In classical selfish mining, the attacker maintains a private lead

$$\Delta = L_{adv} - L_{honest} \geq 1.$$

When the honest chain mines a new block and reduces the lead by one (i.e., $\Delta \rightarrow \Delta - 1$), the attacker strategically releases one previously withheld block to trigger a fork race. The success of this maneuver depends on γ , which determines the fraction of honest miners that follow the attacker’s branch in the resulting tie.

Under the proposed protocol, the effect of this disclosure depends critically on whether the attacker’s privately mined block carries an $n_k_declaration$ consistent with the honest majority.

Case 1: Declaration Alignment

If the attacker’s block carries an $n_k_declaration$ consistent with the collective decision of the honest majority, disclosure induces a fork race analogous to the classical model. However, since consensus priority is determined at the Compact Block layer and Compact Block propagation is not assumed to be more asymmetric than Bitcoin’s Full Block propagation,

$$\gamma_{new} \leq \gamma_{Bitcoin}.$$

Thus, even in the canonical Δ –reduction disclosure scenario, the attacker’s strategic advantage is not increased relative to the classical setting.

Case 2: Declaration Misalignment

If the attacker’s withheld block carries an $n_k_declaration$ inconsistent with the honest majority’s validated state, then upon disclosure the block fails declaration validation and is rejected at the consensus boundary. Honest nodes refuse to extend that branch, and no effective tie-breaking race occurs.

For that disclosure event, the effective propagation advantage collapses:

$$\gamma_{new} \approx 0.$$

Hence, the canonical “lead reduction $\rightarrow$ strategic release” maneuver is either bounded by classical γ (alignment case) or neutralized at the validation layer (misalignment case).

Threshold Preservation

In both cases,

$$\gamma_{new} \leq \gamma_{Bitcoin}.$$

Let α^* satisfy

$$R_{SM}(\alpha^*, \gamma) = \alpha^*.$$

Since the propagation advantage parameter is not increased under the proposed protocol, the profitability threshold is weakly preserved:

$$\alpha_{new}^* \geq \alpha_{Bitcoin}^*.$$

Therefore, the protocol maintains, at minimum, the same level of resilience to selfish mining as Nakamoto Consensus, while eliminating any auxiliary advantage arising from Full Block withholding or payload-level propagation asymmetry.

4.2.3. Structural Disadvantage of False $n_k_declaration$ Attacks

We analyze the scenario in which an adversary deliberately falsifies the $n_k_declaration$ field. For concreteness, assume that the network is currently mining the block at height $n + k$. The adversary mines block with a declaration opposite to the honest majority’s state by declaring “False” when the $n - th$ block has been widely propagated, or “True” when it has not. We assume the adversary does not declare “True” for a non-existent block. We compare this attack surface with classical Bitcoin.

Immediate Network Unification in Bitcoin: In Bitcoin, if an adversary mines the $n + k - th$ block first, the network can be unified under the adversary’s chain simply by publishing that block. Since consensus is determined solely by longest-chain PoW priority, the first valid extension at height $n + k$ can immediately redirect honest mining power toward the attacker’s branch. Thus, in Bitcoin, winning a single block race at height $n + k$ is sufficient to unify the network under the attacker’s chain.

Structural Inability to Unify in the Proposed Protocol (Game-Theoretic and Mathematical Analysis of Consensus Stability): In the proposed protocol, this property no longer holds under “False” $n_k_declaration$. Even if the adversary mines the $n + k - th$ block first, the block carries a declaration inconsistent with the honest majority’s validated state. Consequently, honest nodes reject the block at the declaration-validation layer and refuse to extend the attacker’s branch. Therefore, mining a single block at height $n + k$ does not suffice to unify the network. This can be formalized as follows.

Theorem 2 (Immunity to Malicious Declarations).

Assume a set of rational miners who possess the common knowledge that the $n - th$ block is valid and has already been propagated to over 90% of the nodes in the network ($p \geq 0.9$). Under this condition, any $n + k - th$ compact block carrying a malicious false declaration ($n_k_declaration = false$) will be ignored by these rational miners. Consequently, the honest chain remains the unique Nash Equilibrium, and the malicious branch will be orphaned with a probability of at least 0.81.

Proof.

Let B be the fixed block reward and m be the relative hashing power of an individual rational miner i . An attacker (minority $q \leq 0.1$) publishes a $n + k - th$ block with a malicious declaration ($n_k_declaration = false$), gaining a $z = 1$ block lead. Knowing that $p \geq 0.9$ of the network's hash power is already aligned with the valid $n - th$ block, miner i evaluates two strategies: **Maintenance** (S_T) or **Defection** (S_F).

1. Expected Utility of Maintenance ($E[G_T]$) :

Miner i recognizes that the honest majority, possessing $\geq 90\%$ of the total hash rate, will attempt to reorganize the chain. The probability $P_{success}$ of the honest majority mining two successive blocks ($n + k$ and $n + k + 1$) to reclaim the lead is:

$$P_{success} = p^2 \geq (0.9)^2 = 0.81$$

The expected gain for miner i on this chain is:

$$E[G_T] = m \cdot B \cdot P_{success} \geq 0.81 \cdot m \cdot B$$

2. Expected Utility of Defection ($E[G_F]$):

If miner i defects to the malicious branch, the reward is realized only if the 90% honest majority fails to reorganize. The probability of this failure is at most $(1 - P_{success})$.

Thus, the expected gain is:

$$E[G_F] = m \cdot B \cdot (1 - P_{success}) \leq (1 - 0.81) \cdot m \cdot B = 0.19 \cdot m \cdot B$$

3. Nash Equilibrium Condition:

Given the miners' knowledge of the 90% propagation, they compare the utilities:

$$E[G_T]/E[G_F] \geq 0.81 \cdot m \cdot B / 0.19 \cdot m \cdot B \approx 4.26 > 1$$

Since the expected return for remaining on the true chain is significantly higher, and miners know their peers are also rational and informed, S_T becomes the only logical choice.

4. Convergence:

The shared knowledge of 90% propagation ensures that rational miners do not fear isolation on the true chain. Instead, they fear the 0.81 probability of their block being orphaned if they join the false branch. Therefore, the false declaration is collectively ignored. $\square$

A critical question arises in the implementation of the aforementioned theorem: How can an individual node in a decentralized network attain high confidence that the $n - th$ block has indeed reached the 90% threshold? In a distributed system without a global observer, establishing such a consensus on propagation state is non-trivial. However, the proposed protocol provides a robust empirical mechanism for this verification: the analysis of the arrival interval between block signals.

To preserve the structural simplicity of the Proof-of-Work (PoW) mechanism, the protocol avoids hard-coded temporal constraints. Instead, it utilizes the arrival interval $\Delta T = T_{CB(n+k)} - T_{FB(n)}$, where $T_{CB(n+k)}$ is the arrival timestamp of the $n + k - th$ compact block and $T_{FB(n)}$ is the arrival timestamp of the $n - th$ full block, as a critical heuristic for individual nodes to estimate the network-wide propagation state. If ΔT significantly exceeds the known network diameter (the time required for a block to traverse the majority of the network), a miner attains high confidence that the $n - th$ block has achieved Common Knowledge among the 90% majority.

This temporal gap serves as a decentralized proof of propagation; as ΔT increases, the probability that a significant portion of the hashing power remains unaware of block n diminishes exponentially. This confidence solidifies the $E[G_T]$ expectation, incentivizing rational miners to ignore the false $n + k$ declaration and compete for the true $n + k$ reward, effectively liquidating the attacker's investment.

The proposed mechanism leverages the economic "skin in the game" of miners rather than moral imperatives. By rendering false declarations economically suicidal under high propagation conditions—verifiable via ΔT —the protocol ensures that the "Longest Chain Rule" naturally converges toward the truth. In instances where ΔT is negligible, implying a genuine propagation delay or a race condition, the system gracefully treats the exclusion of block n as a standard orphan event. This allows the chain to progress toward the highest cumulative PoW without compromising systemic stability. Ultimately, the integration of temporal heuristics into the rational decision-making process ensures that truth remains the most profitable path in the decentralized landscape.

Consequently, the following operational requirements are identified for system integrity.

Parameter k Calibration: The value of k must be configured sufficiently large to allow the $n - th$ full block to be fully disseminated across the network, ensuring that the majority of hashing power can validate the block before the arrival of the $n + k - th$ declaration.

Mandatory Rapid Propagation: Mining nodes that release a $n - th$ compact block must propagate the corresponding full block data as expeditiously as possible to the entire network; any delay in data availability may be misinterpreted as a malicious false declaration, leading to the deliberate orphaning of the block by rational peers.

Requirement of Private Chain Domination: After mining the $n + k - th$ block with a "False" declaration, the adversary is effectively isolated. Since honest nodes will not build upon that chain, the attacker must continue extending the private branch alone. To successfully override the honest chain, the adversary must mine a longer chain $n + k + j$ ($j > 0$) before the honest majority extends their branch.

This implies:

- ♦ No propagation advantages
- ♦ No honest hash redirection
- ♦ Pure private PoW race

Reduction to conditional private fork attack: Since no honest miner contributes to the attacker's chain under "False" declaration, the adversary must independently accumulate sufficient PoW to surpass the honest chain. Unlike Bitcoin, where winning a single block race at height $n + k$ can immediately unify the network, the proposed protocol structurally prevents such unification under declaration inconsistency.

Forgery is a strictly dominated conditional private fork attack that yields no strategic advantage over original private fork attack as follows.

After block $n - 1$ is published, the adversary begins mining a private branch in secret. The attack is declared successful only if the adversary achieves a lead of at least one block strictly after the mining event corresponding to block n . Thus, even if the adversary mines block n immediately after publication, this event alone does not constitute success.

This additional constraint modifies the stochastic structure of fork competition in a mathematically nontrivial way.

Let

$$\alpha \in (0, 1/2)$$

denote the adversary's relative hash power, and define

$$\beta = 1 - \alpha$$

as the relative hash power of the honest network.

Define the state variable

$$\Delta_t = a_t - h_t,$$

where:

a_t is the length of the adversary's private chain extension beyond block $n - 1$,

h_t is the length of the honest network's public chain extension beyond block $n - 1$.

The process evolves as a discrete-time biased random walk:

$$\Delta_{t+1} = \begin{cases} \Delta_t + 1, & \text{with probability } \alpha \\ \Delta_t - 1, & \text{with probability } \beta \end{cases}$$

with initial condition

$$\Delta_0 = 0.$$

The attack succeeds if

$$\exists t \geq 2 \text{ such that } \Delta_t \geq 1.$$

Importantly,

$$\Delta_t = 1$$

is excluded from the success set, since it corresponds to the adversary mining block n immediately after the publication of block $n - 1$.

Theorem 3 (Exact Success Probability)

Under the above model, the probability of attack success is

$$P_{success} = \alpha^2(2 + 1/(1 - \alpha)) \text{ for } \alpha < 1/2.$$

Proof.

We condition on the first mining event after the publication of block $n - 1$.

Case 1: The adversary mines first (probability α)

Then

$$\Delta_1 = 1,$$

which does not constitute success.

Two subcases:

1. The adversary mines again (probability α):

$$1 \rightarrow 2,$$

yielding immediate success. Contribution: α^2 .

2. The honest network mines (probability β):

$$1 \rightarrow 0.$$

From state 0, the probability of ever reaching +1 is

$$\alpha/\beta.$$

Hence this branch contributes

$$\alpha\beta\alpha/\beta = \alpha^2.$$

Total contribution from Case 1:

$$2\alpha^2.$$

Case 2: The honest network mines first (probability β)

Then

$$\Delta_1 = 1.$$

The probability that a biased random walk starting from -1 ever reaches $+1$ is

$$(\alpha/\beta)^2.$$

Thus, the contribution is

$$\beta(\alpha/\beta)^2 = \alpha^2/\beta.$$

Summation

$$P_{success} = 2\alpha^2 + \alpha^2/\beta = \alpha^2(2 + 1/(1 - \alpha)) < \alpha/(1 - \alpha). \quad \square$$

Consequently, “False” $n_k_declaration$ Attack constitutes a conditional private fork attack, which is inherently weaker than other well-known attacks on Bitcoin, such as the original private fork attack or selfish mining.

4.2.4. Strategic Inertness of the k –Block Window under Compact Block Consensus

The $n_k_declaration$ mechanism introduces a temporal gap between Compact Block consensus and the final validation of the corresponding Full Block. Specifically, the validity of the Full Block F_n is determined only when the chain reaches height $(n + k)$. During this interval, the Compact Block chain continues to grow based solely on Proof-of-Work validity. During this delayed validation window, an invalid Full Block may be linked to a Compact Block and temporarily remain in the chain until the final validation point is reached. In such cases, the invalidity of F_n is detected only at height $(n + k)$, at which point the corresponding transaction-level state transition is rejected. Importantly, however, this temporary persistence does not allow the adversary to permanently alter the ledger state. The consensus protocol ensures that only valid Full Blocks contribute to state transitions once the validation point is reached. Therefore, the adversary can at most influence the temporary ordering of Compact Blocks within the delayed validation window, but cannot force invalid transactions into the final ledger state.

Consequently, the delayed validation window introduces a bounded temporal uncertainty without compromising the long-term safety of the blockchain state.

We examine the strategic space available to an adversary during the k -block interval spanning from height n to $n + k - 1$, under the assumption that Compact Block consensus has already been established for this range. Assume that the network is currently mining the block at height $n + k$. By construction, the preceding k blocks—namely those at heights $n, n + 1, \dots, n + k - 1$ —have already propagated and been accepted in their Compact Block form by the honest majority. Each Compact Block includes the PoW commitment, compressed transaction identifiers, and the associated declaration fields. Although Full Block data verification may still be pending within the prescribed k -window, consensus over Compact Block objects themselves has already been formed.

This structural property severely constrains the adversary’s strategic options.

First, the adversary cannot modify or replace any of the blocks within this interval without invalidating the corresponding PoW commitments. Since the Compact Blocks themselves have already been accepted by the honest majority, altering their contents would require producing an alternative chain with strictly greater cumulative PoW. Thus, any attempt to rewrite this interval reduces to a classical majority (51%) attack.

Second, withholding Full Blocks within the k -window does not confer a strategic advantage. Mining priority is determined exclusively at the Compact Block layer. Honest nodes extend the chain upon acceptance of valid Compact Blocks. If the corresponding Full Block is not delivered before the expiration of the k -window, the block is deterministically declared invalid (“False”), resulting in forfeiture of its reward. Therefore, payload-level withholding cannot create a private lead.

Third, the only remaining action available to an adversary during this interval is to manipulate the $n_k_declaration$ at height $n + k$. However, as shown previously, a declaration inconsistent with the honest majority leads to immediate isolation of the adversary’s branch and eliminates any propagation advantage. The attack thereby collapses into an isolated hash-power race, again equivalent to a classical majority attack.

In fact, the proposed protocol does not fundamentally alter the Nakamoto consensus mechanism. Instead, it introduces a temporal separation between chain ordering and Full Block validation. At height n , the network only establishes a provisional ordering of block n . The final consensus regarding block n is effectively reached at height $n + k$, when sufficient subsequent PoW blocks collectively confirm its validity. In this sense, the mechanism can be interpreted as shifting the effective consensus point from n to $n - k$ relative to the current chain tip. The $n_k_declaration$ therefore represents a collectively produced statement of validity, backed by accumulated PoW. Only declarations supported by the majority of hash power survive, making the process effectively a PoW-weighted voting mechanism. Consequently, the protocol preserves the fundamental principle of blockchain consensus: majority agreement secured by proof-of-work.

4.2.5. Concrete Adversarial Scenarios and Their Structural Limitations

To further illustrate the robustness of the proposed protocol, we analyze several concrete adversarial strategies and demonstrate how each is structurally constrained under Compact Block consensus.

A. Indiscriminate False Declaration Attacks

Attack Scenario: An attacker, despite having received a valid $n - th$ Full Block, declares $n_k_declaration = False$ in the $(n + k) - th$ block to exclude certain blocks or intercept rewards.

Mitigation: Under the "Truth-Observation Principle," the majority of honest nodes have already received the $n - th$ Full Block. Upon receiving the $(n + k) - th$ Compact Block, nodes cross-reference the declaration with their local data status. If the declaration contradicts locally verified data, the block is classified as an invalid declaration and rejected without further propagation.

Economic Penalty: The attacker forfeits the entire block reward and expends hashing resources without compensation. Since “False” declarations provide no structural advantage while imposing certain loss, rational miners are economically disincentivized from adopting such behavior.

Of course, such an attack would become feasible if the adversary controlled a majority of the network’s hash power. In that case, however, the situation simply reduces to the classical 51% attack and therefore lies outside the threat model considered here. Furthermore, a strategy in which the adversary persistently issues $n_k_declaration$ with the value “False” is not economically rational. Since the declaration mechanism applies symmetrically to all blocks, after k subsequent blocks the same rule inevitably applies to blocks produced by the adversary itself. Consequently, a miner that systematically declares “False” would eventually invalidate its own blocks as well.

From a game-theoretic perspective, such a strategy is therefore self-defeating and incentive-incompatible, as it destroys the economic value of the adversary's own mining output. As a result, even a strategic adversary has a structural incentive to eventually issue "True" declarations for valid blocks, which fundamentally limits the sustainability of indiscriminate false-declaration strategies. In this sense, the protocol embeds an intrinsic economic stabilization mechanism against indiscriminate declaration attacks.

B. Data Hiding and Partitioning Attacks

This strategy attempts to induce incorrect declarations by temporarily withholding the $n - th$ Full Block and subsequently releasing it to trigger a partition or deep reorganization.

Attack Scenario: An adversary intentionally withholds the $n - th$ Full Block, forcing honest nodes to declare "False" due to data unavailability. The attacker then releases the data later, attempting to split the consensus and trigger a deep reorganization.

Economic Self-Destruction: Even if the attacker eventually releases the $n - th$ Full Block, if the honest majority who proceeded with the "False" declaration possesses superior hash power, the attacker's $n - th$ block will be definitively excluded (Off) from the ledger. The attacker suffers a total loss of the capital and energy expended on mining the $n - th$ block, with zero block rewards. The attempt to partition the network results not in a successful fork, but in the attacker's own economic catastrophe.

Strategic Timing Obstacles: For a partitioning attack to be effective, the adversary must release the hidden data with surgical precision. Given that Compact Blocks propagate at near instantaneous speeds, predicting or hitting this micro-second window across a global network is physically unfeasible. Any slight deviation in timing leads to either the "False" chain becoming too dominant to overturn or the honest nodes receiving the data in time to declare "True". This extreme sensitivity to timing, combined with the risk of total reward loss, renders data withholding attacks a losing proposition for rational actors.

Failure of Network Partitioning through Full Block Withholding: Suppose an adversary attempts to hide the Full Block corresponding to height n . Such an attack implicitly assumes that the adversary aims to force honest miners to produce blocks with $n_k_declaration = false$, while the adversary itself later issues $n_k_declaration = true$ and attempts to mine the $n + k$ block first in order to override the honest majority. As discussed earlier, this strategy already exposes the adversary to significant risk.

First, the only entity capable of withholding the Full Block at height n is the miner who actually produced that block. Suppose this miner sets $n_k_declaration = true$ and also succeeds in mining the $n + k$ block before the honest network. The probability of such a sequence of events decreases rapidly and is roughly proportional to the square of the adversary's hash power. Furthermore, if the adversary continues to withhold the Full Block at height n , the network does not split into multiple competing honest groups. Instead, the network effectively separates into exactly two parts: the adversary and the honest network. Since honest nodes never received the Full Block at height n , they cannot validate a block at height $n + k$ containing $n_k_declaration = true$. Consequently, honest nodes reject both the compact and full versions of such a block and do not relay them further, thereby preventing any unnecessary bandwidth wastage.

As a result, the Full Block containing $n_k_declaration = true$ remains known only to the adversary who withheld the block, leaving the adversary effectively isolated from the rest of the network. Therefore, although the adversary might attempt to partition the network, the intended objective of splitting the honest miners into multiple competing groups ultimately fails.

Ultimately, the adversary is forced into a hash-power race against the honest majority, a competition destined for failure as long as the majority remains honest. Even if the adversary manages to produce a temporarily longer chain, they face a structurally insurmountable disadvantage. This inherent strategic failure can be analyzed as follows.

C. Ghost Chain Attacks (Chain Deception via Data Withholding)

Attack Scenario: An adversary attempts to hijack the network by constructing a deceptive longer chain, declaring "True" for the $n - th$ block despite having withheld the actual Full Block data from the network.

Mitigation via Mandatory Verification: The primary safeguard is the principle of "Conditional Acceptance." Honest nodes strictly refuse to switch to any competing chain, regardless of its PoW height, until they have fully downloaded and verified the $n - th$ Full Block data.

Physical Deterrence via Propagation Bottleneck: Even if an attacker secretly builds a longer chain and eventually releases the hidden data to induce a reorganization (Reorg), they face a fatal physical barrier. The time required to transmit large scale Full Block data across the global network acts as a "Physical Speed Limit." During this transmission period, honest nodes who

are already mining on the legitimate, verified chain continue to extend their chain further. This ensures that a deceptive chain, burdened by its own delayed data transfer, cannot outpace the growth of the honest chain.

Analysis: Since forged data is filtered out instantly by hash integrity checks, the only viable path for an attacker is to transmit genuine data. However, the physical latency of this transmission prevents the "Ghost Chain" from ever achieving the dominance required to overturn the network's consensus. Consequently, any deceptive chain lacking data is inevitably isolated and pruned from the network.

4.3. Efficient Utilization of Proof-of-Work under Post-Validation Rejection

In the conventional consensus mechanism of Bitcoin, a block that is later determined to be invalid is entirely discarded from the blockchain. As a consequence, the Proof-of-Work expended in producing that block contributes nothing to the cumulative security of the chain. Formally, if a mined block B_i is invalid, the effective security contribution of its computational work becomes

$$S(B_i) = 0.$$

In contrast, the proposed architecture separates the consensus object from the transaction execution data by distinguishing between a Compact Block and its corresponding Full Block. The Compact Block participates in the consensus process and establishes the ordering of blocks, while the Full Block contains the transaction data required for state transitions and validation.

As long as the Compact Block satisfies the consensus rules and its PoW is valid, it remains permanently embedded in the chain regardless of whether the corresponding Full Block subsequently passes transaction-level validation. In such cases, even if the Full Block F_i is rejected due to transaction-level inconsistencies, the Compact Block C_i remains part of the canonical chain and its PoW continues to contribute to the accumulated chain security:

$$S(C_i) > 0.$$

Accordingly, the effective security of the chain can be expressed as

$$S_{chain} = \sum_{i=1}^n S(C_i),$$

where every valid Compact Block contributes to the accumulated PoW regardless of subsequent transaction-level validity outcomes.

This architectural separation ensures that the computational work performed by miners is not entirely lost even when transaction-level validation failures occur. In contrast to Bitcoin, where invalid blocks lead to a complete loss of the associated PoW, the proposed mechanism preserves the security contribution of mined blocks whenever the consensus-level structure remains valid.

Consequently, the network's hash power is utilized more efficiently, and the cumulative PoW securing the chain more accurately reflects the total computational effort expended by miners. In other words, the PoW produced by honest miners continues to reinforce the security of the block-ordering layer even when transaction-level data later proves to be invalid. The preserved PoW secures the temporal ordering of blocks, even if the associated transaction state later proves invalid.

5. PERFORMANCE ANALYSIS

As with Bitcoin, absolute security cannot be formally proven under fully asynchronous networks. Consequently, security is expressed as a parameter-dependent probabilistic guarantee. Bitcoin's fork convergence analysis implicitly relies on the fundamental network assumption that the propagation delay is significantly smaller than the block interval, formally expressed as Propagation delay $\ll$ Block interval. Similarly, the proposed scheme implicitly assumes that the propagation delay of a Full Block is significantly smaller than the $(k - i)$ -block validation interval. Consequently, a vast majority of honest nodes are expected to receive and verify the Full Block prior to the issuance of the corresponding $n_k_declaration$. The selected parameters (10MB block size, 600-second block interval, $k = 6$ and $i = 2$) are conservatively established within the previously derived safe operational region. This ensures that the probability $P(\text{all honest nodes know } F_n \text{ before } n + k)$, representing the likelihood that every honest node recognizes the Full Block F_n before the $(n + k)$ -th block is mined, remains sufficiently high. These architectural considerations are discussed as follows.

5.1. Compact Block Capacity and Efficiency Analysis

The efficiency of the proposed Compact Block is governed by the bit-length of compressed UTXO IDs and the resulting probability of collisions. While conventional Compact Block Relay (CBR) suffers from critical propagation stalls when a collision occurs requiring complex request response cycles to resolve the ambiguity, our protocol remains robust. In our system, a collision merely results in "over-freezing," where a few additional UTXOs are temporarily locked, posing no threat to the consensus continuity or propagation speed.

5.1.1. Mathematical Analysis of Collision Probability and Over Freezing

To determine the optimal ID size, we analyze the collision expectations based on a 10MB block size.

System Parameters: A 10MB block contains approximately 40,000 transactions. With an average of 2.5 inputs per transaction, the number of UTXOs to be compressed per block is 100,000.

Mempool Scale: Assuming a mempool of 500,000 pending transactions, the total number of UTXO IDs monitored is 1,250,000.

Collision Expectation: The expected number of collisions in a set of n elements with an ID space of $H = 2^b$ is approximately $E = n^2 / (2H)$. Considering that a collision involves at least two UTXOs, the expected count of UTXOs participating in collisions is approximately $2 \times E$. We analyze ID sizes of 4, 5, 6, 7 and 8 bytes. To maintain a negligible over-freezing rate even in high throughput environments, a 6-byte (48-bit) ID is selected as the optimal balance between compression and collision resistance (Table 4). As presented in Table 4, when using 6-byte compressed hashes, the expected number of UTXOs involved in hash collisions within the mempool is less than 1. Even in the rare event of a collision, "over-freezing" only affects an insignificant fraction of UTXOs in the mempool, ensuring that honest transactions are virtually unaffected.

Table 4: Collision Expectation

ID Size (Bytes)	Bit-length (b)	ID Space (H)	Expected UTXOs in Collision ($n = 1.25 \times 10^6$)
4 Bytes	32-bit	4.29×10^9	364
5 Bytes	40-bit	1.10×10^{12}	1.42
6 Bytes	48-bit	2.81×10^{14}	0.0055
7 Bytes	56-bit	7.20×10^{16}	0.0000217
8 Bytes	64-bit	1.84×10^{19}	0.0000000847

5.1.2. Comparison with Conventional CBR

Unlike conventional CBR which compresses entire transactions, our protocol compresses individual UTXOs used as inputs.

ID Count: Historical Bitcoin data shows an average of 2.5 inputs per transaction. Thus, our Compact Block contains approximately 2.5 times more IDs than CBR.

Propagation Superiority: Although the data volume is slightly higher than CBR, our Compact Block achieves faster effective propagation. Conventional CBR often stalls due to missing transactions in the local mempool, requiring "Request-Response" cycles. By utilizing a list of UTXO IDs for immediate validation, our Compact Block eliminates round trips and achieves performance equivalent to having a 100% synchronized mempool.

5.1.3. Independence from Mempool Synchronization

The proposed protocol fundamentally decouples block propagation from mempool state. Upon receiving a valid Compact Block, a node can immediately proceed to mine the next block. This is because the block's validity for mining purposes is determined solely by the verification of its header (Proof-of-Work) and the objective consistency of its $n_k_declaration$.

The mempool is not consulted for transaction reconstruction at this stage. This design choice grants the protocol perfect robustness against mempool asynchrony, a critical weakness in conventional CBR. However, an indirect, second order influence exists on network wide efficiency. The subsequent, Full Block propagation phase (which is decoupled from the mining cycle) can leverage fast relay protocols like CBR [25-29]. The efficiency of this Full Block propagation is, as in any system, dependent on mempool synchronization. Poor mempool synchronization can slow the diffusion of the 10MB Full Block data across the network. This delay only affects the background data synchronization. In our protocol, the mempool's role is demoted from a critical path dependency to a mere background efficiency optimizer.

5.1.4. Security against Denial of Service (DoS) via UTXO compressed ID

The use of a 6-byte compressed UTXO ID provides a total address space of 2^{48} , which equals 281,474,976,710,656 (approximately 2.8×10^{14}) unique values. Given that the current global UTXO set of Bitcoin is approximately 10^8 , the available ID space is more than 1000,000 times larger than the number of existing UTXOs. Crucially, the probability that an adversary can occupy or influence the entire address space through a single block (containing 10^5 UTXO IDs) is only $10^5/2^{48} \approx 3.55 \times 10^{-10}$. This vast sparsity ensures that the probability of an adversary successfully mapping or exhausting the ID space within a single block is negligible.

5.2. Network Propagation Simulation, Parameter Validation, and Throughput Scaling

This section provides a comparative simulation of propagation delays and validates the security parameter k under scaled block sizes, ultimately demonstrating the protocol's ability to increase transaction throughput without compromising security. Specifically, to ensure the robustness of our analysis, the values used in this simulation were conservatively assumed with an integrated safety margin, rendering them more restrictive than typical real-world data. Additionally, these timings serve only as a relative guideline rather than absolute values, given the inherently dynamic and unpredictable nature of network states.

5.2.1. Compact Block Propagation Delay Simulation and Comparison

To ground our analysis in realistic network dynamics, we model propagation delays based on established Bitcoin network research [30-36] and the fundamental differences in protocol design.

A. Proposed Protocol: Compact Block Propagation

By scale-adjusting our model, we evaluate the performance of Compact Blocks across various Full Block sizes. By employing a 6-byte compressed UTXO ID for each input, the size of our Compact Block remains fixed relative to the number of inputs. For instance, a Compact Block corresponding to a 10MB Full Block (containing 100,000 inputs) is a fixed 600 KB message. The propagation time (δ_{CB}) of the Compact Block is dominated by network latency and bandwidth. Based on models for Bitcoin block propagation, the time required for a Compact Block to reach 90% of nodes in a well-connected Bitcoin-like network is simulated as follows.

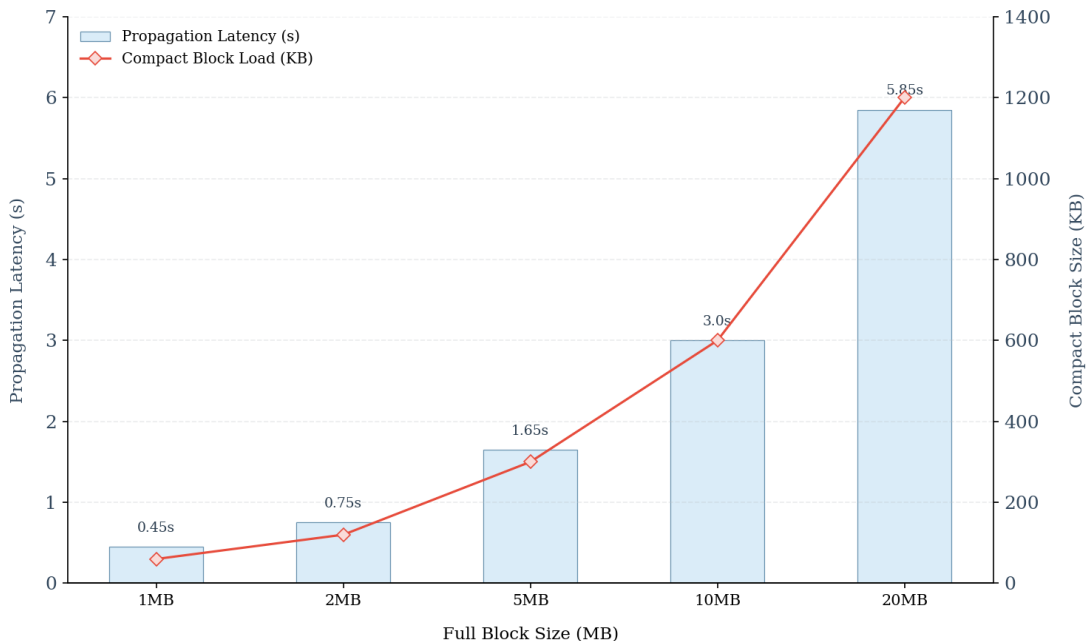


Figure 1. Performance Analysis: Propagation Delay with Compact Block Scaling

In our scheme, Compact Blocks do not require mempool-dependent recovery, ensuring that the propagation delay remains highly stable and independent of the mempool state.

B. Bitcoin with CBR: 1 MB Block Propagation

The propagation of a 1 MB block using CBR is a two-phase stochastic process: transmission of a small Compact Block (~50 KB), and recovery of missing transactions via separate requests. The total time (δ_{BTC}) is heavily dependent on mempool synchronization between the relaying and receiving nodes. We simulate δ_{BTC} for the block to reach 90% of nodes under varying mempool sync rates, drawing from empirical studies of CBR performance.

Table 5: Simulated 90% Network Propagation Delay for a 1 MB Block (Bitcoin with CBR)

Mempool Synchronization Rate	Simulated Propagation Delay (δ_{BTC})	Primary Bottleneck
70%	~12s	Regular recovery cycles
90%	~5s	Occasional recovery
95%	~3s	Near optimal CBR performance

C. Comparison and Implication

Consistent with the Bitcoin network, the block finding process in our scheme is modeled as a Poisson process with a rate of $\lambda = 1/600s^{-1}$. Under this model, the probability of a fork occurring during the Compact Block propagation delay δ_{CB} is given by

$$P_{CB} = 1 - e^{-\lambda\delta_{CB}} \approx \lambda\delta_{CB}.$$

In contrast, the fork probability for a standard Bitcoin block is expressed as

$$P_{BTC} = 1 - e^{-\lambda\delta_{BTC}} \approx \lambda\delta_{BTC}.$$

From this, it can be inferred that if $\delta_{CB} < \delta_{BTC}$, then $P_{CB} < P_{BTC}$. While the peak performance of optimal CBR (~3 seconds at 95% sync) is comparable to our Compact Block's propagation (~3 seconds) for a 10MB Full Block (Figure 1), our protocol eliminates the performance variance and the risk of extreme delays associated with low mempool sync rates.

The proposed system delivers consistently low latency propagation, effectively decoupling the critical path for consensus from the unpredictable state of the mempool. This results in an effective network hesitation period (δ_{CB}) that is as good as or better than Bitcoin's under optimal conditions (δ_{BTC}) and significantly superior under non-ideal, real-world conditions. In our scheme, the Full Block size can be configured up to a maximum of 10MB, as the condition $\delta_{CB} \leq \delta_{BTC}$ is no longer satisfied once the block size exceeds this threshold (Figure 1). It should be noted, however, that this is based on a considerably conservative set of assumptions.

Since each block discovery is an independent event (memoryless property of the Poisson process), the probability of a fork extending to a specific depth k is roughly the probability of k consecutive "near-simultaneous" mining events occurring before the network synchronizes.

The probability of a fork reaching depth k is:

$$P(\text{depth} \geq k) \approx P_{BTC}^k$$

Assuming a propagation delay of $\delta_{BTC} = 3$ seconds, which yields a single block fork probability of $P_{BTC} = 3/600 = 0.005$, the probability of a fork reaching a depth of $k \geq 2$ is calculated as $P(\text{depth} \geq 2) \approx (0.005)^2 = 0.000025$ (or 1 in 40,000 blocks). This represents a statistically negligible probability, implying that at a standard generation rate of 144 blocks per day, such an event is expected to occur approximately once every nine months. From this, we establish $i = 2$ as the default confirmation threshold to ensure general network stability.

5.2.2. Full Block Propagation and Security Analysis of Parameter r

A. Full Block Propagation Simulation

The 10 MB Full Block propagation can utilize fast relay protocols like CBR. We simulate its 90% propagation time (δ_{FB}) under the same mempool sync conditions, scaling the delays from the 1 MB model to account for the larger data volume and longer recovery phases.

Table 6: Simulated 90% Network Propagation Delay for 10 MB Full Block

Mempool Synchronization Rate	Simulated Propagation Delay (δ_{FB})	Primary Bottleneck
70%	~90s	Regular recovery cycles
90%	~35s	Occasional recovery
95%	~20s	Near optimal CBR performance

B. Probability of k Blocks Mined Before Full Block Propagation

A security critical event occurs if $r(=k-i)$ blocks are found before the Full Block propagates to the network majority, potentially leading to a n_k declaration fork. We model block finding as a Poisson process with rate $\lambda = 1/600s^{-1}$.

The number of blocks $N(t)$ generated within time interval t follows a Poisson distribution with parameter λt :

$$N(t) \sim \text{Poisson}(\lambda t)$$

The probability of exactly r blocks being mined within time t is:

$$P(N(t) = r) = \frac{e^{-\lambda t} (\lambda t)^r}{r!}, r = 0, 1, 2, \dots$$

The probability of mining at least r blocks within time t is the complement of the probability of mining fewer than r blocks:

$$P(N(t) \geq r) = \sum_{i=r}^{\infty} \frac{e^{-\lambda t} (\lambda t)^i}{i!} = 1 - \sum_{i=0}^{r-1} \frac{e^{-\lambda t} (\lambda t)^i}{i!} = 1 - e^{-\lambda t} \sum_{i=0}^{r-1} \frac{(\lambda t)^i}{i!}$$

In our scheme, the fork rate induced by the Full Block propagation delay δ_{FB} is denoted by

$$P_{FB} = P(N(\delta_{FB}) \geq r).$$

Table 7 presents the upper bounds of δ_{FB} satisfying $P_{FB} < \epsilon$ corresponding to different values of r .

Table 7: Critical Time Values (δ_{FB}) satisfying ($P_{FB} < \epsilon$)

r	$\epsilon = 0.001$	$\epsilon = 0.0001$
1	$\delta_{FB} < 0.6s$	$\delta_{FB} < 0.06s$
2	$\delta_{FB} < 27.24s$	$\delta_{FB} < 8.52s$
3	$\delta_{FB} < 114.32s$	$\delta_{FB} < 51.7s$
4	$\delta_{FB} < 257.13s$	$\delta_{FB} < 139.07s$

5	$\delta_{FB} < 443.62s$	$\delta_{FB} < 266.67s$
6	$\delta_{FB} < 664.26s$	$\delta_{FB} < 428.24s$
7	$\delta_{FB} < 912.20s$	$\delta_{FB} < 618.23s$
8	$\delta_{FB} < 1182.48s$	$\delta_{FB} < 832.18s$

C. Validation of r and Throughput-Security Proposition

The simulation and probability analysis lead to two key conclusions:

Parameter Security: As shown in Table 7, the upper bound of the propagation delay that maintains a negligible probability of a n_k declaration fork is maximized when $r = 8$. This validates $r = 8$ as a highly conservative and secure choice. While $r = 8$ offers an immense mathematical safety margin, our analysis demonstrates that $r = 4$ is already functionally secure, with the $\delta_{FB} < 257.13s$ (139.07s) and probability of a fork being as low as 10^{-3} (10^{-4}).

Therefore, we have opted for $r = 4$ ($i = 2, k = i + r = 6$) as the standard parameter to significantly accelerate transaction finality without compromising practical security. This choice strikes an optimal balance between maintaining a robust security posture and enhancing the responsiveness of the network. Nevertheless, as a more conservative alternative for enhanced safety, the system can also be configured with an 8-byte UTXO compressed ID and $r = 8$ ($i = 3, k = 11$). In this configuration, the network achieves an even higher level of cryptographic security and fork resistance.

Throughput Scaling without Security Loss: The core result is that increasing the block size by a factor of 10 (from 1 MB to 10 MB) to increase transaction throughput does not necessitate an increase in the security parameter r . Specifically, the $k = 6$ ($i = 2, r = 4$) configuration is designed to offer a transaction finality time equivalent to the traditional 6-confirmation standard in Bitcoin. The elegant decoupling of consensus (n_k declaration via Compact Block) from data availability (Full Block propagation) allows the system to scale data capacity while maintaining the same, or even improved, resilience against propagation-triggered forks compared to the base Bitcoin network ($P_{CB} \leq P_{BTC}, P_{FB} \leq P_{BTC}$). The security of the consensus layer remains intact and independent of the scaling of the data layer.

The security analysis of Bitcoin is predicated on the implicit assumption that the block propagation delay is significantly shorter than the block mining interval. Similarly, the security arguments presented in Section 4 for our proposed scheme are based on the following implicit assumptions:

- ♦ The propagation delay of a Full Block is significantly shorter than the time required to mine r blocks.
- ♦ The propagation time of a Compact Block corresponding to a single block is shorter than that of a standard 1MB Full Block in the Bitcoin network.

These assumptions are fundamentally guaranteed by the parameter configurations established above (10MB block size, 600-second block interval, $k = 6$ and $i = 2$).

5.2.3. Throughput Calculation and Scaling

The selection of system parameters is critical for balancing security and throughput. For simulation and safety margin maximization, we established $k = 6$ (evaluation window), $i = 2$ (Full Block propagation gate), and a 10MB Block Size while maintaining the Bitcoin-standard 600-second interval. Assuming an average transaction size of 250 bytes, a 10MB block accommodates approximately 40,000 transactions. The transaction throughput (TPS) is calculated as follows:

$$\text{Throughput}(TPS) = \frac{10,000,000 \text{ bytes} / 250 \text{ bytes per TX}}{600 \text{ seconds}} = \frac{40,000}{600} \approx 66.67 \text{ TPS}$$

This represents a nearly ten-fold increase compared to the standard Bitcoin protocol (~ 7 TPS) without requiring modifications to the mining frequency.

5.3. Solving the Blockchain Trilemma: Decoupling Throughput from Propagation

The proposed *n_k-declaration* protocol presents a novel architecture that redefines the relationship between the three axes of the classic blockchain trilemma—Scalability, Security, and Decentralization. It demonstrates that a fundamental re-engineering of the consensus propagation interface allows for a breakthrough where system throughput is no longer directly constrained by the physics of network latency, enabling genuine scaling without the traditional sacrifices.

5.3.1. Scalability: Asynchronous Pipeline and Unconstrained Block Size

Scalability is achieved through a structural decoupling, creating an asynchronous, two-stage pipeline.

Stage 1 - Consensus & Chain Growth (Compact Block): The 600 KB Compact Block, containing only essential commitments, propagates in near real time (~3 seconds). This decouples the progress of the chain (via PoW) and the formation of consensus (via *n_k-declaration*) from the transfer of bulk transaction data. The chain can grow at the theoretical maximum speed permitted by the PoW difficulty.

Stage 2 - Data Availability (Full Block): The 10 MB Full Block, containing the complete transaction set, propagates in the background on a separate timeline. Its arrival and validation are necessary for delayed final state settlement but are not on the critical path for block creation and consensus.

This architecture breaks the direct link between block size and orphan risk. Consequently, the block size can be increased to 10 MB (enabling ~40,000 transactions per block, a 10x increase in TPS) based solely on sustainable network bandwidth, not on security concerns regarding chain stability.

5.3.2. Security: Preserved and Enhanced by Full Hash Power and the *r*-Barrier

The protocol not only preserves but structurally enhances the security model of Nakamoto Consensus.

100% Efficient Honest Hash Power: The rapid propagation of the Compact Block ensures that all honest nodes regardless of their geographic location or connection speed receive the new block header and can begin mining on top of it almost simultaneously. This eliminates the "network handicap" that delays honest hash power in traditional PoW, effectively increasing the relative power of the honest mining majority against attackers.

The *r*-Block Cryptographic Barrier: The security parameter *r* creates a deterministic, time-based confirmation barrier. As proven in Section 5.2.2, even with 10 MB blocks, the probability $P(N(\delta_{FB}) \geq r)$ is negligible (Table 7). The *n_k-declaration* mechanism and fork resolution rules further ensure that any attempt to create an alternative history is both economically irrational and algorithmically doomed.

5.3.3. Decentralization: Leveling the Mining Playing Field

Decentralization is fortified by removing systemic advantages tied to network infrastructure.

Stable Mining Interval (600s): The protocol maintains Bitcoin's 10-minute block interval, ensuring the mining reward schedule and economic incentives remain stable and predictable.

Elimination of Propagation Induced Orphan Risk: By decoupling the consensus signal (Compact Block) from the data payload (Full Block), a node's chance to successfully propagate a block and have it accepted is no longer a function of its bandwidth to upload 10 MB of data before competitors. A well-connected node in a data center and a modestly connected home miner face nearly identical conditions in the race to propagate the decisive Compact Block. This democratizes mining participation and protects the network from centralizing forces driven by edge network advantages.

Our protocol demonstrates that scalability (throughput), security (immutability, attack resistance), and decentralization (permissionless mining) can be advanced simultaneously when the core constraint which is the synchronous binding of propagation to consensus is architecturally dissolved.

6. DISCUSSION

The proposed mechanism for Compact Blocks and asynchronous consensus primarily focused on significantly improving network throughput. This approach offers the following scalability and areas for future research.

First, this proposal has primarily centered on the scenario of increasing the block size. However, the protocol's fundamental "Latency Hiding" mechanism can be equally applied to reducing the block interval (mining frequency). For instance, by keeping block sizes similar to the current standard but decreasing the interval to five minutes or less, the *k*-block isolation and compact propagation method can maintain transaction finality while suppressing the orphan block rate. Consequently, further granular

research is required to determine the optimal configurations balancing the mining interval, block size, and the values of k and i to maximize overall network efficiency.

Second, to induce the miner of the $(n + k)$ -th block to make a more responsible declaration regarding the validity of the n -th block ($n_k_declaration$), an economic incentive mechanism could be introduced. A potential approach is to share a portion (e.g., 10%) of the n -th block's reward with the $(n + k)$ -th block miner, contingent on the n -th block being finalized as valid. This provides a strong motivation for the $n + k$ miner to actively verify data availability and declare truthfully. The precise calibration of these reward ratios and the resulting impact on network stability remain compelling avenues for future exploration.

Furthermore, while our scheme presents a significant architectural leap, it is important to acknowledge that two aspects might hypothetically lower efficiency, though their practical impact in real-world scenarios is minimal:

Impact of the k -parameter on UTXO expenditure frequency: In our proposed scheme, we set the parameter $k = 6$, thereby prohibiting the expenditure of a UTXO within six blocks of its creation. Given that six blocks are the standard threshold for transaction finality in Bitcoin, it is highly unconventional for typical users to reconsume a UTXO within such a short duration. Consequently, this constraint imposes no significant burden on the vast majority of users while effectively enhancing network security.

Efficiency in scenarios with high input-to-transaction ratios: While the efficiency of our proposed scheme may decrease in hypothetical scenarios where the number of UTXO inputs per block significantly outweighs the number of transactions, such occurrences are practically negligible in real world environments. Empirically, the vast majority of Bitcoin transactions utilize only one or two inputs. This trend reflects not only a cost optimization strategy to minimize transaction fees but also a user preference for simplicity in asset management, aligning with the principle of cognitive economy.

7. CONCLUSION

We proposed an asynchronous block validation and logical isolation protocol to resolve the block propagation delay, a fundamental bottleneck to scalability in PoW based blockchains. The system proposed herein strictly inherits Bitcoin's security model while leveraging architectural innovation to achieve the following three core conclusions.

Firstly, by decoupling block generation from the final validity confirmation using an $n + k$ post declaration mechanism, the physical latency associated with large block propagation is entirely hidden within the subsequent block's PoW computation time. This demonstrates that network idle time can be eliminated, expanding throughput (TPS) to the limits of the physical bandwidth.

Secondly, we ensured logical independence between blocks through UTXO Compressed ID-based Mempool Freezing and an extended maturity design. This structurally insulates the invalidation of the n -th block from affecting the validity of the chain from $n + 1$ onwards, fundamentally eliminating the risk of cascading failures inherent in previous serial structures.

Thirdly, this protocol establishes a robust security principle: "allowing false negatives (misunderstandings) but architecturally blocking false positives (fraud)". The propagation mechanism and mandatory data existence verification rules provide optimal convergence efficiency for honest nodes while imposing asymmetric barriers (a propagation delay and prohibitive economic penalties) on attackers.

In conclusion, this research realized a high-performance consensus structure capable of processing large-scale blocks without compromising the simplicity and decentralization of Bitcoin. The practical implications of this model are twofold:

First, by fundamentally fortifying the throughput and efficiency of the Layer 1 mainnet, this protocol serves as a powerful catalyst when coupled with Layer 2 solutions such as the Lightning Network or Rollups. The enhanced data availability of the base layer provides a more robust foundation, enabling these upper-layer technologies to achieve exponentially greater scalability.

Second, by faithfully inheriting Bitcoin's core architecture, our protocol ensures seamless compatibility with existing mining hardware and infrastructure. This allows for the immediate utilization of the massive global PoW mining assets without requiring costly hardware transitions, facilitating a pragmatic and cost-effective path toward a high-performance blockchain ecosystem.

REFERENCES

- [1] K. Croman et al., "On Scaling Decentralized Blockchains," in Financial Cryptography and Data Security (LNCS 9604), pp. 106-125, 2016.
- [2] Q. Zhou, H. Huang, Z. Zheng and J. Bian, "Solutions to Scalability of Blockchain: A Survey," in IEEE Access, vol. 8, pp. 16440-16455, 2020.
- [3] M. J. M. Chowdhury, A. Colman, M. A. Kabir, J. Han and P. D. Sinderen, "A Survey on Blockchain Scalability," in Journal of Network and Computer Applications, vol. 127, pp. 35-58, 2019.
- [4] A. Hafid, A. S. Hafid and M. Samih, "Scaling Blockchains: A Comprehensive Survey," in IEEE Access, vol. 8, pp. 125304-125320, 2020.
- [5] T. A. Alghamdi, R. Khalid and N. Javaid, "A Survey of Blockchain Based Systems: Scalability Issues and Solutions, Applications and Future Challenges," in IEEE Access, vol. 12, pp. 79626-79651, 2024.
- [6] G. A. F. Rebello, G. F. Camilo, L. A. C. de Souza, et al., "A Survey on Blockchain Scalability: From Hardware to Layer-Two Protocols," in IEEE Communications Surveys & Tutorials, vol. 26, no. 4, pp. 2411-2458, 2024.
- [7] D. Khan, L. T. Jung and M. A. Hashmani, "Systematic Literature Review of Challenges in Blockchain Scalability," in Applied Sciences, vol. 11, no. 20, 9372, 2021.

-
- [8] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," Technical Report, 2008. [Online]. Available: bitcoin.org
- [9] J. A. Garay, A. Kiayias and N. Leonardos, "The Bitcoin Backbone Protocol: Analysis and Applications," in *Advances in Cryptology – EUROCRYPT* 2015, vol. 9057, pp. 281-313, 2015.
- [10] I. Eyal, A. E. Gencer, E. G. Sirer and R. Van Renesse, "Bitcoin-NG: A Scalable Blockchain Protocol," in *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI '16)*, pp. 45-59, 2016.
- [11] A. Sompolinsky and Y. Zohar, "Accelerating Bitcoin's Transaction Processing: BCH and GHOST," in *International Conference on Financial Cryptography and Data Security (FC)*, pp. 37-53, 2015.
- [12] A. Gangwal, H. R. Gangavalli and A. Thirupathi, "A Survey of Layer-two Blockchain Protocols," in *Journal of Network and Computer Applications*, vol. 209, p. 103539, Jan. 2023.
- [13] J. Poon and T. Dryja, "The Bitcoin Lightning Network: Scalable Off-chain Instant Payments," in *Proc. Lightning Labs*, San Francisco, CA, USA, vol. 9, Jan. 2016, p. 14.
- [14] M. Corallo, "BIP 152: Compact Block Propagation," *Bitcoin Improvement Proposals*, 2016.
- [15] A. P. Ozisik, G. Bissias and B. N. Levine, "Graphene: Efficient Interactive Set Reconciliation Applied to Blockchain Propagation," in *ACM SIGCOMM Computer Communication Review*, vol. 49, no. 1, pp. 16-27, 2019.
- [16] M. Corallo, "The FIBRE Network," [Online]. Available: <https://bitcoinfibre.org>
- [17] Y. Sompolinsky, Y. Lewenberg and A. Zohar, "SPECTRE: A Fast and Scalable Cryptocurrency Protocol," in *Cryptology ePrint Archive*, Report 2016/1159, 2016.
- [18] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert and P. Saxena, "A Secure Sharding Protocol for Open Blockchains," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Vienna, Austria, Oct. 2016, pp. 17-30.
- [19] M. B. Saif, S. Migliorini and F. Spoto, "A Survey on Data Availability in Layer 2 Blockchain Rollups: Open Challenges and Future Improvements," in *Future Internet*, vol. 16, no. 9, art. no. 315, Aug. 2024.
- [20] A. Gervais, G. O. Karamé, K. Wüst, V. Glykantzis, H. Ritzdorf and S. Capkun, "On the Security and Performance of Proof-of-Work Blockchains," in *Proc. ACM CCS*, pp. 3-16, 2016.
- [21] X. Li, P. Jiang, T. Chen, X. Luo and Q. Wen, "A Survey on the Security of Blockchain Systems," in *Future Generation Computer Systems*, vol. 107, pp. 841-853, Jun. 2020.
- [22] M. Saad et al., "Exploring the Attack Surface of Blockchain: A Comprehensive Survey," in *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 1977-2008, 2020.
- [23] I. Eyal and E. G. Sirer, "Majority is Not Enough: Bitcoin Mining is Vulnerable," in *Proc. FC*, pp. 436-454, 2014.
- [24] A. Sapirshstein, Y. Sompolinsky and Z. Zohar, "Optimal Selfish Mining Strategies in Bitcoin," in *Proc. FC*, pp. 515-532, 2016.
- [25] L. Zhang, T. Wang and S. C. Liew, "Speeding up block propagation in Bitcoin network: Uncoded and coded designs," in *Computer Networks*, vol. 206, art. no. 108791, Apr. 2022.
- [26] J. Mišić, V. B. Mišić and X. Chang, "On the Benefits of Compact Blocks in Bitcoin," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020.
- [27] C. Zhao, S. Zhang, T. Wang and S. C. Liew, "Bodyless block propagation: TPS fully scalable blockchain with pre-validation," in *Future Generation Computer Systems*, vol. 163, art. no. 107516, Feb. 2025.
- [28] Z. H. Chin, V. M. Baskaran, C. K. Tan, I. K. T. Tan and T. T. V. Yap, "DAP-CBR: enhancing Bitcoin block propagation efficiency using dynamic compact block relay's prefilling of transactions," in *The Journal of Supercomputing*, vol. 81, art. no. 77, Jan. 2025.
- [29] A. P. Ozisik, G. Andresen, G. Bissias, A. Houmansadr and B. Levine, "Graphene: A New Protocol for Block Propagation Using Set Reconciliation," in *Data Privacy Management, Cryptocurrencies and Blockchain Technology*, Cham, Switzerland: Springer, 2017, pp. 420-428.
- [30] Y. Shahsavari, K. Zhang and C. Talhi, "A Theoretical Model for Block Propagation in Bitcoin-like Networks," in *Proc. IEEE CNS*, pp. 1-9, 2019.
- [31] C. Decker and R. Wattenhofer, "Information Propagation in the Bitcoin Network," in *Proc. IEEE P2P*, pp. 1-10, 2013.
- [32] M. Fadhil, G. Owenson and M. Adda, "A Bitcoin model for evaluation of clustering to improve propagation delay in Bitcoin network," in *Proc. IEEE Int. Conf. Comput. Sci. Eng. (CSE)*, Aug. 2016, pp. 468-475.
- [33] M. Fadhil, G. Owenson and M. Adda, "Locality based approach to improve propagation delay on the Bitcoin peer-to-peer network," in *Proc. IFIP/IEEE Symp. Integr. Netw. Service Manage. (IM)*, May 2017, pp. 556-559.
- [34] M. Fadhil, G. Owenson and M. Adda, "Proximity awareness approach to enhance propagation delay on the bitcoin peer-to-peer network," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jun. 2017, pp. 2411-2416.
- [35] W. Bi, H. Yang and M. Zheng, "An accelerated method for message propagation in blockchain networks," 2018, arXiv:1809.00455. [Online]. Available: <https://arxiv.org/abs/1809.00455>
- [36] Y. Aoki and K. Shudo, "Proximity neighbor selection in blockchain networks," 2019, arXiv:1906.00719. [Online]. Available: <https://arxiv.org/abs/1906.00719>